1. Create a customized Docker image using a Dockerfile.

Objective: Create a custom Nginx image using Amazon Linux as the base image, run a container, and test the application.

Implementation steps

1. Connected to the Amazon Linux EC2 instance using SSH.
2. Installed Docker, started the Docker service, and verified that Docker was running.
3. Created the project directory and navigated into it.
4. mkdir -p ~/docker-challenge/custom-nginx

cd ~/docker-challenge/custom-nginx

5. Created an index.html file containing the custom webpage content.
6. Created a file named Dockerfile with the following instructions:
7. FROM amazonlinux:latest
8. MAINTAINER Poojith
9. RUN dnf install -y nginx && dnf clean all
10. COPY index.html /usr/share/nginx/html/index.html
11. EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]

12. Built the custom Docker image.

sudo docker build -t custom-nginx:v3 .

13. Verified that the image was created successfully.

sudo docker images

14. Created and started a container using host port 8085, mapped to container port 80.

sudo docker run -d --name custom-nginx-container -p 8085:80 custom-nginx:v3

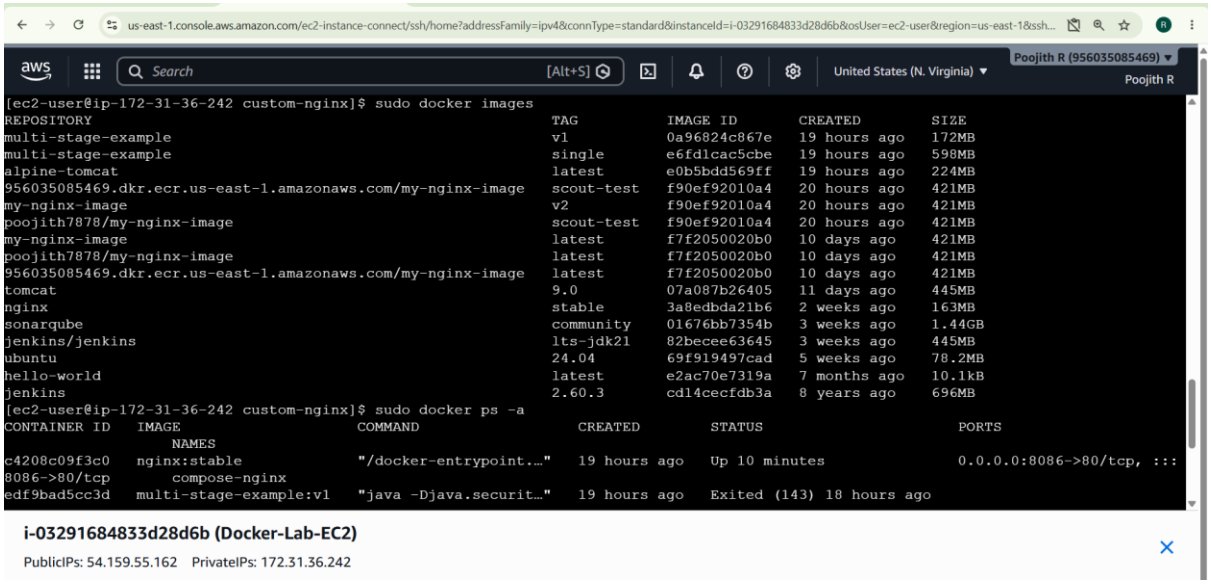
15. Verified the running container.

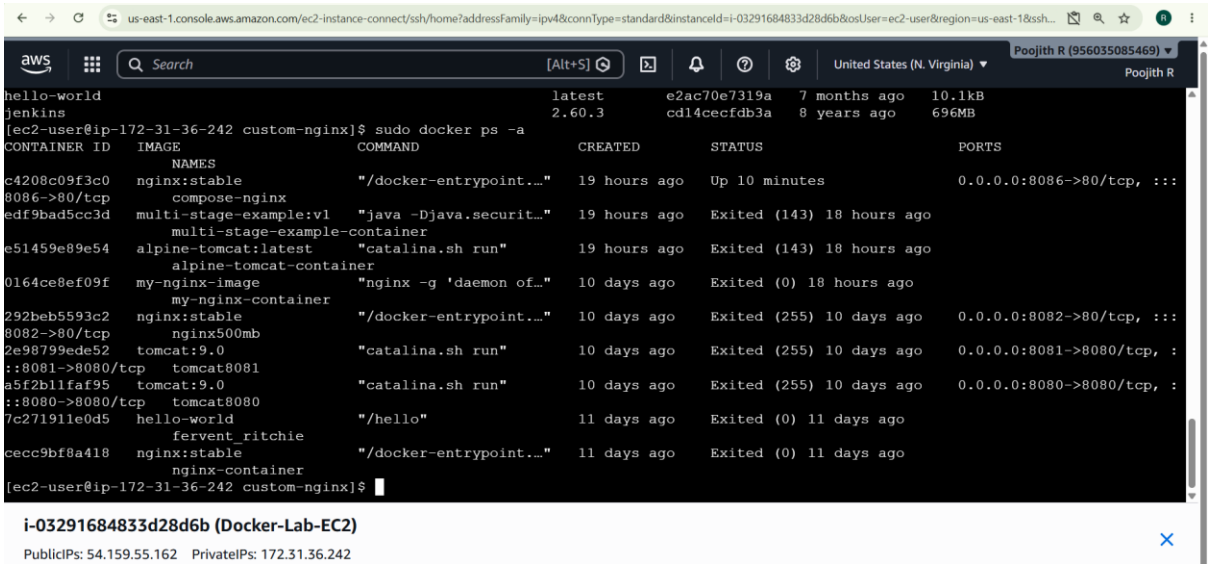
sudo docker ps

16. Tested the application using the browser URL http://EC2_PUBLIC_IP:8085 or the local command:

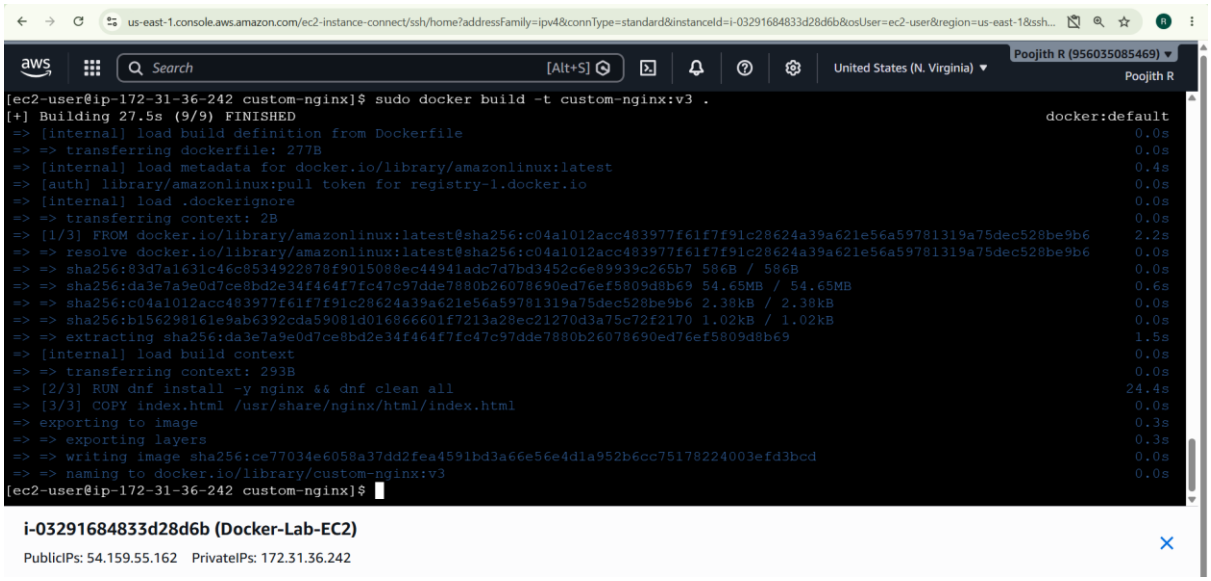
curl http://localhost:8085

17. Confirmed that the custom webpage was accessible.

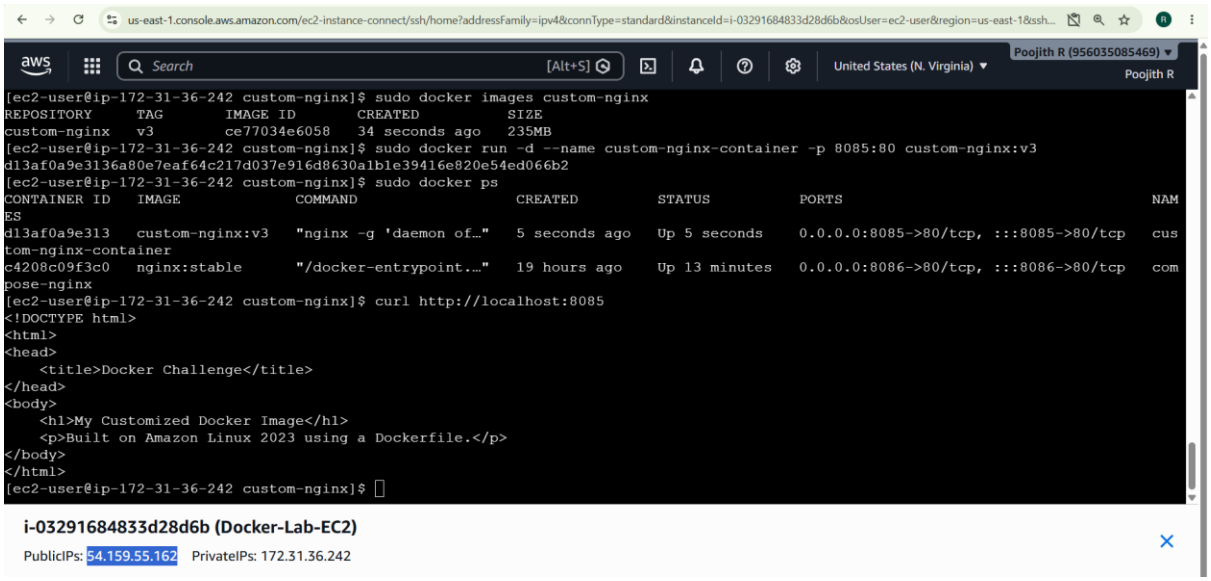





```
[ec2-user@ip-172-31-36-242 custom-nginx]$ ls -l
total 8
-rw-r--r--. 1 ec2-user ec2-user 179 Oct 24 06:42 Dockerfile
-rw-r--r--. 1 ec2-user ec2-user 195 Oct 24 06:40 index.html
[ec2-user@ip-172-31-36-242 custom-nginx]$ cat Dockerfile
FROM amazonlinux:latest
MAINTAINER Poojith
RUN dnf install -y nginx && dnf clean all
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
[ec2-user@ip-172-31-36-242 custom-nginx]$
```









2. Push the image to Docker Hub.

Objective: Upload the custom Nginx image to Docker Hub so that it can be stored and downloaded from a remote registry.

Implementation steps

1. Created or accessed the Docker Hub account and repository.
2. Logged in to Docker Hub from the EC2 terminal.

sudo docker login

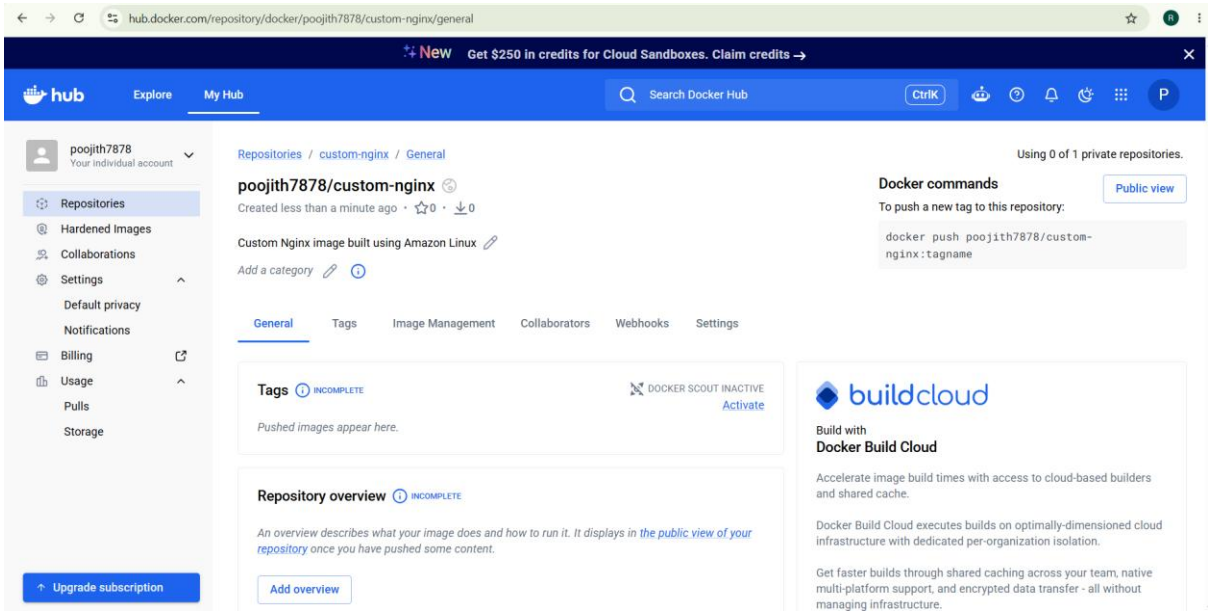
3. Tagged the existing custom image with the Docker Hub repository name.

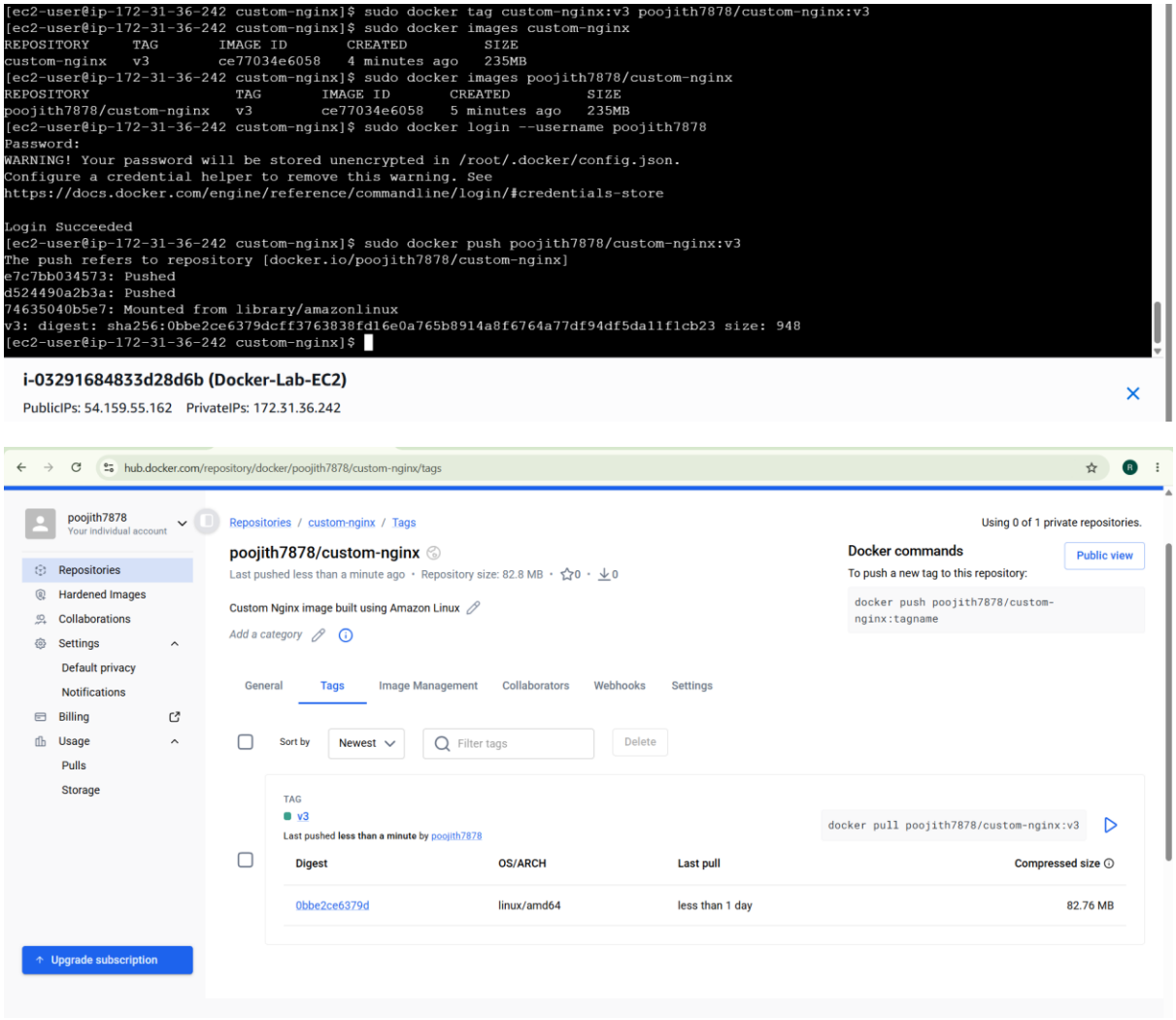
sudo docker tag custom-nginx:v3 poojith7878/custom-nginx:v3

4. Pushed the image to Docker Hub.

sudo docker push poojith7878/custom-nginx:v3

5. Waited for the upload to complete and verified that the push succeeded.
6. Opened the Docker Hub repository and confirmed that the v3 tag was available.





3. Push the same image to Amazon ECR.

Objective: Upload the existing custom Nginx image to Amazon Elastic Container Registry (ECR).

Implementation steps

1. Opened the AWS Management Console and navigated to Amazon ECR.
2. Used the custom-nginx ECR repository in the us-east-1 region.
3. Verified that the AWS CLI was installed and that the EC2 instance had the required IAM permissions.
4. Configured the AWS region.

aws configure set region us-east-1

5. Logged in to ECR using the AWS-provided authentication token.

aws ecr get-login-password --region us-east-1 | sudo docker login --username AWS --password-stdin 956035085469.dkr.ecr.us-east-1.amazonaws.com

6. Tagged the existing image for the ECR repository.

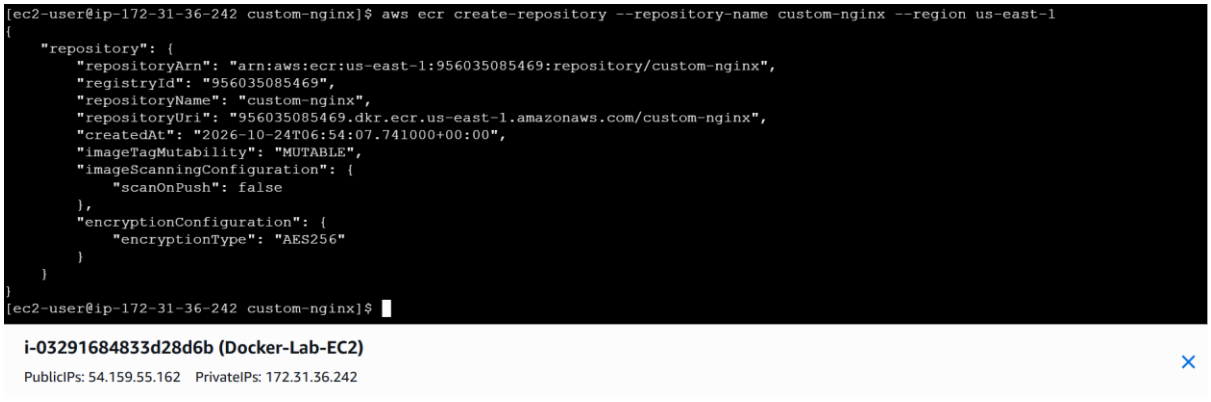
sudo docker tag custom-nginx:v3 956035085469.dkr.ecr.us-east-1.amazonaws.com/custom-nginx:v3

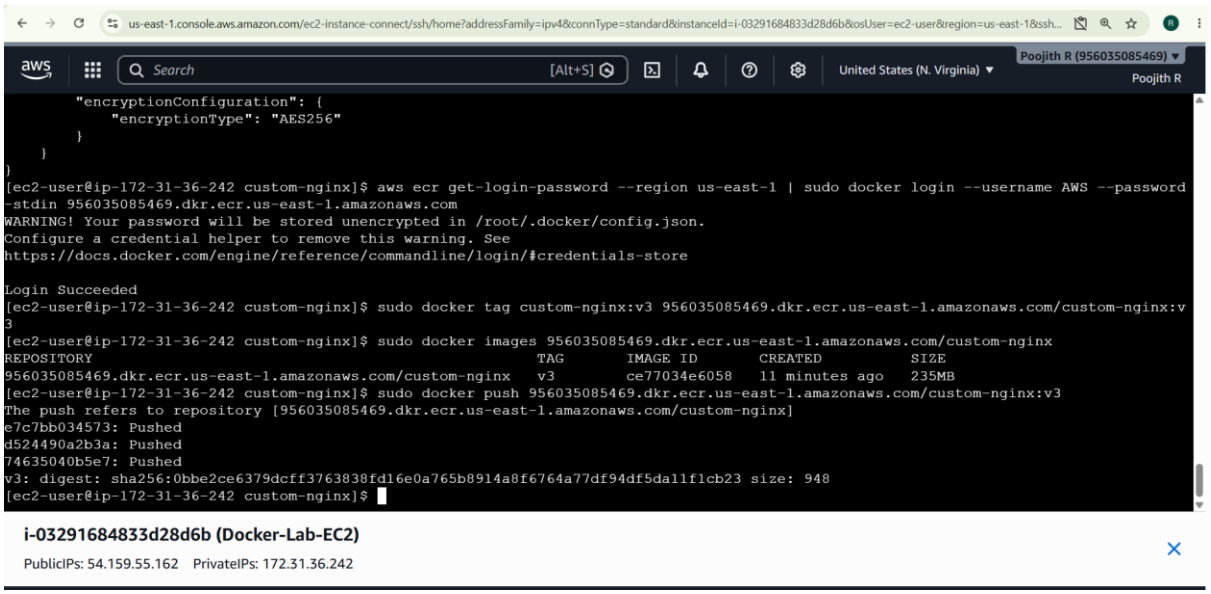
7. Pushed the image to ECR.

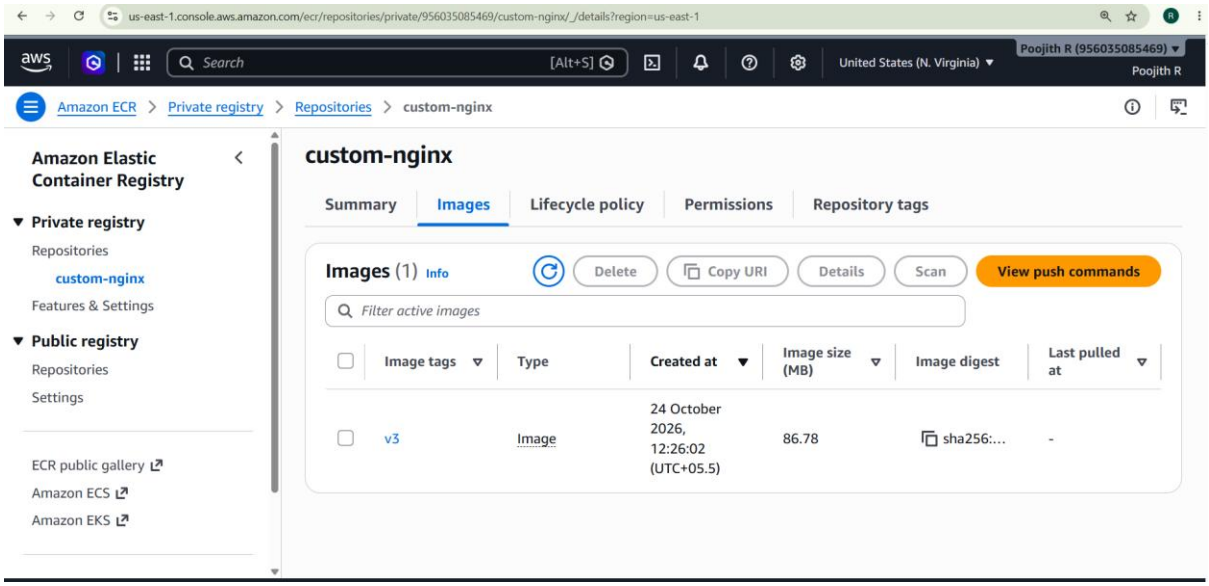
sudo docker push 956035085469.dkr.ecr.us-east-1.amazonaws.com/custom-nginx:v3

8. Opened Amazon ECR in the AWS Console.

9. Verified that the custom-nginx repository contained the v3 image tag.







4. Provision one EC2 instance using Terraform and install Jenkins.

Objective: Use Terraform to provision an EC2 instance and install Jenkins on it.

Implementation steps

1. Connected to the Docker lab EC2 instance and verified that Terraform and the AWS CLI were installed.
2. terraform -version

aws --version

3. Verified AWS access and the configured region.

aws sts get-caller-identity

4. Created a project directory for the Terraform configuration.
5. mkdir -p ~/docker-challenge/terraform-jenkins

cd ~/docker-challenge/terraform-jenkins

6. Created the Terraform configuration file named main.tf.

nano main.tf

7. Configured the AWS provider, Amazon Linux AMI, security group, and EC2 instance resources in the Terraform file.
8. Configured the security group to allow the required access:
    - SSH — TCP port 22, restricted to the administrator's IP.
    - Jenkins — TCP port 8080, restricted to the required client IP.

9. Configured the EC2 instance with the required key pair, instance type, security group, and user-data commands to install Java and Jenkins.
10. Initialized the Terraform working directory.

```
terraform init
```

11. Validated the Terraform configuration.

```
terraform validate
```

12. Reviewed the resources Terraform planned to create.

```
terraform plan
```

13. Created the EC2 instance and supporting resources.

```
terraform apply
```

14. Reviewed the plan and confirmed the operation when prompted.
15. Opened the AWS EC2 console and verified that the Terraform-Jenkins-Server instance was running.
16. Connected to the Jenkins EC2 instance and verified the Jenkins service.

```
sudo systemctl status jenkins
```

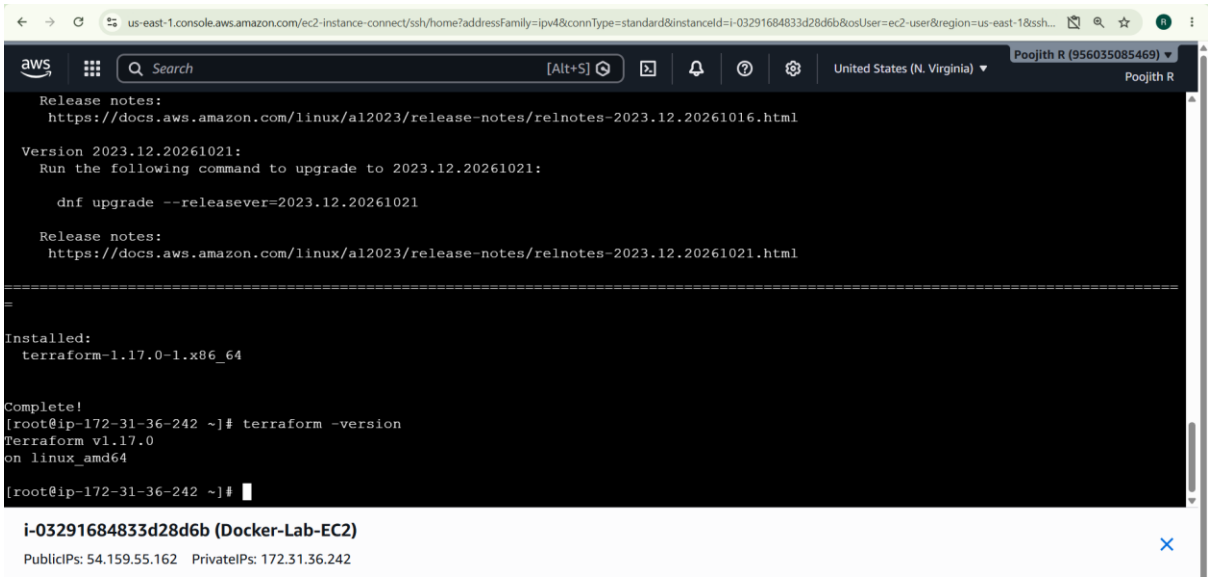
17. Retrieved the initial Jenkins administrator password when required.

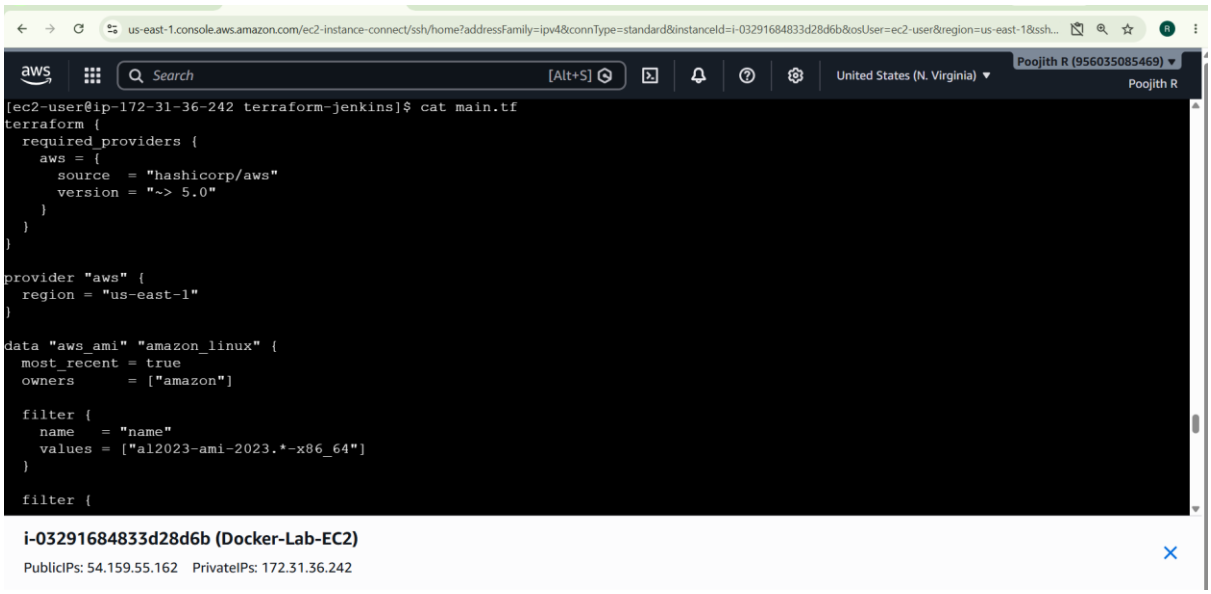
```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

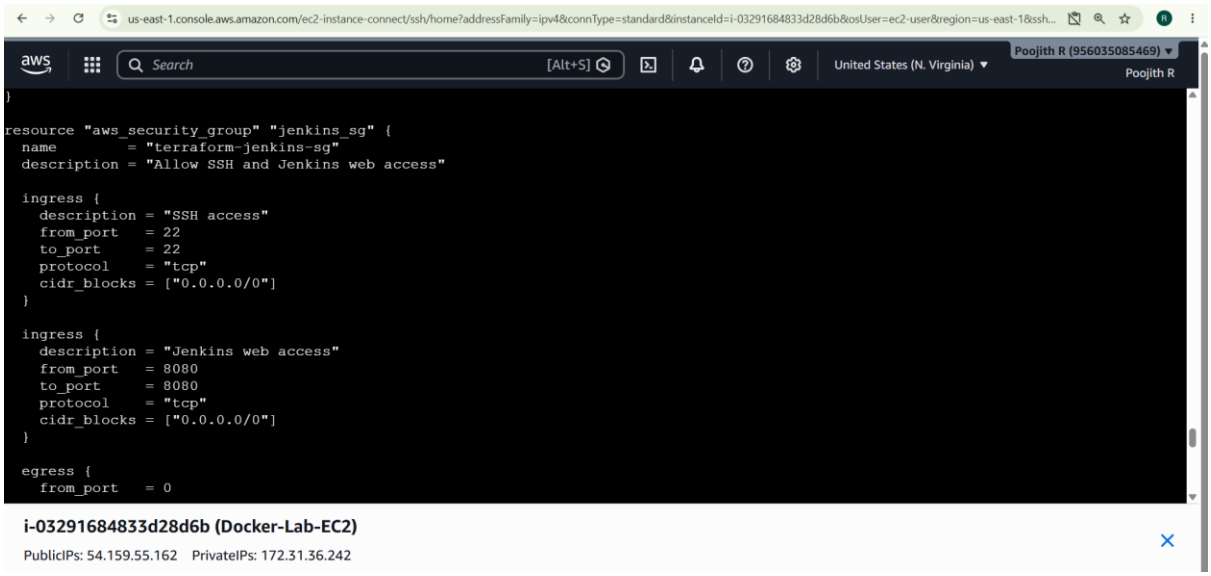
18. Opened Jenkins in the browser using the instance's public IP and port 8080.

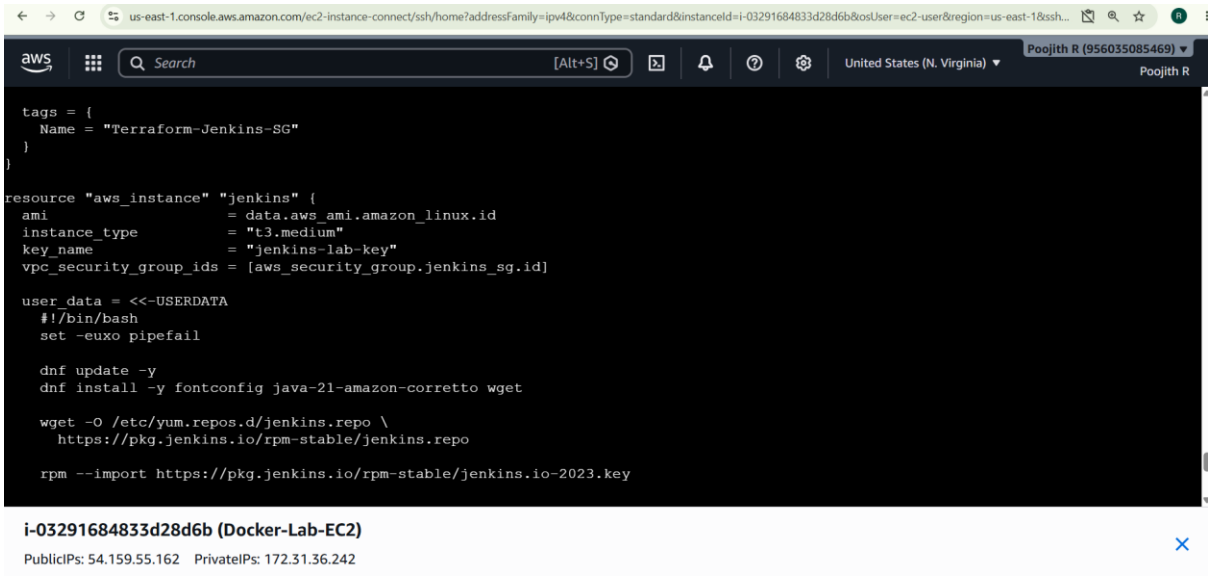
```
http://EC2_PUBLIC_IP:8080
```

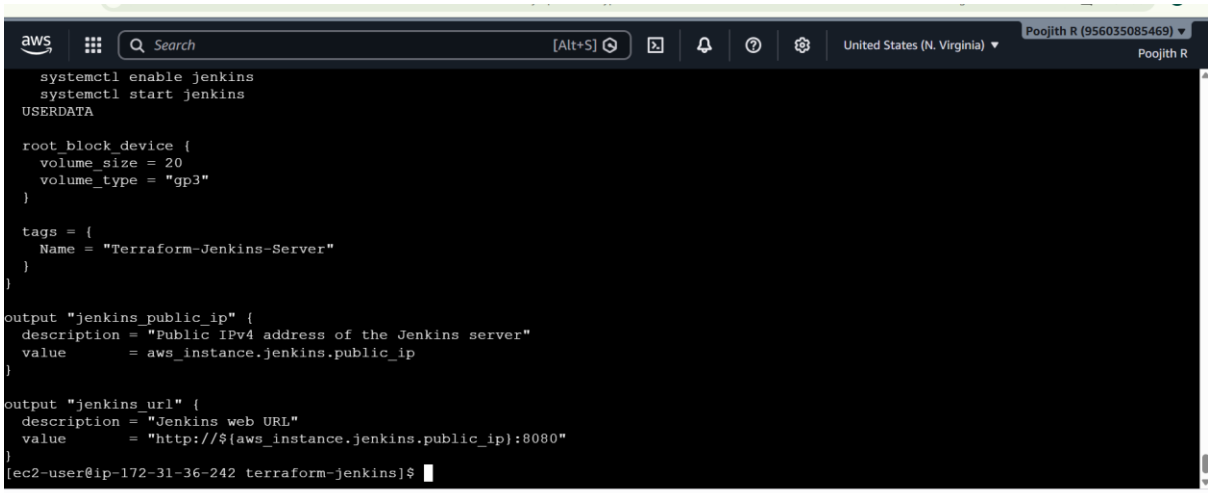
19. Completed the Jenkins setup wizard, installed the required plugins, and created the administrator account.
20. Installed Docker on the Jenkins EC2 instance and verified that the Jenkins user could execute Docker commands.

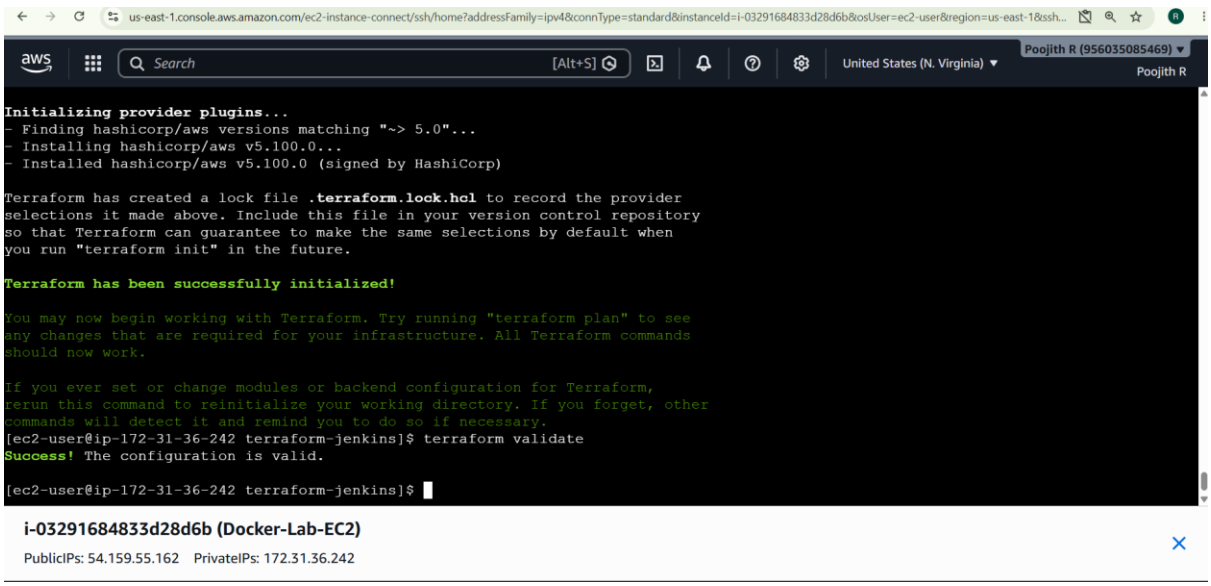

us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?addressFamily=ipv4&connType=standard&instanceId=i-03291684833d28d6b&osUser=ec2-user&region=us-east-1&ssh...
aws
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
Release notes:
https://docs.aws.amazon.com/linux/al2023/release-notes/relnotes-2023.12.20261016.html
Version 2023.12.20261021:
Run the following command to upgrade to 2023.12.20261021:
dnf upgrade --releasever=2023.12.20261021
Release notes:
https://docs.aws.amazon.com/linux/al2023/release-notes/relnotes-2023.12.20261021.html
Installed:
terraform-1.17.0-1.x86_64
Complete!
[root@ip-172-31-36-242 ~]# terraform -version
Terraform v1.17.0
on linux_amd64
[root@ip-172-31-36-242 ~]#
i-03291684833d28d6b (Docker-Lab-EC2)
PublicIPs: 54.159.55.162 PrivateIPs: 172.31.36.242

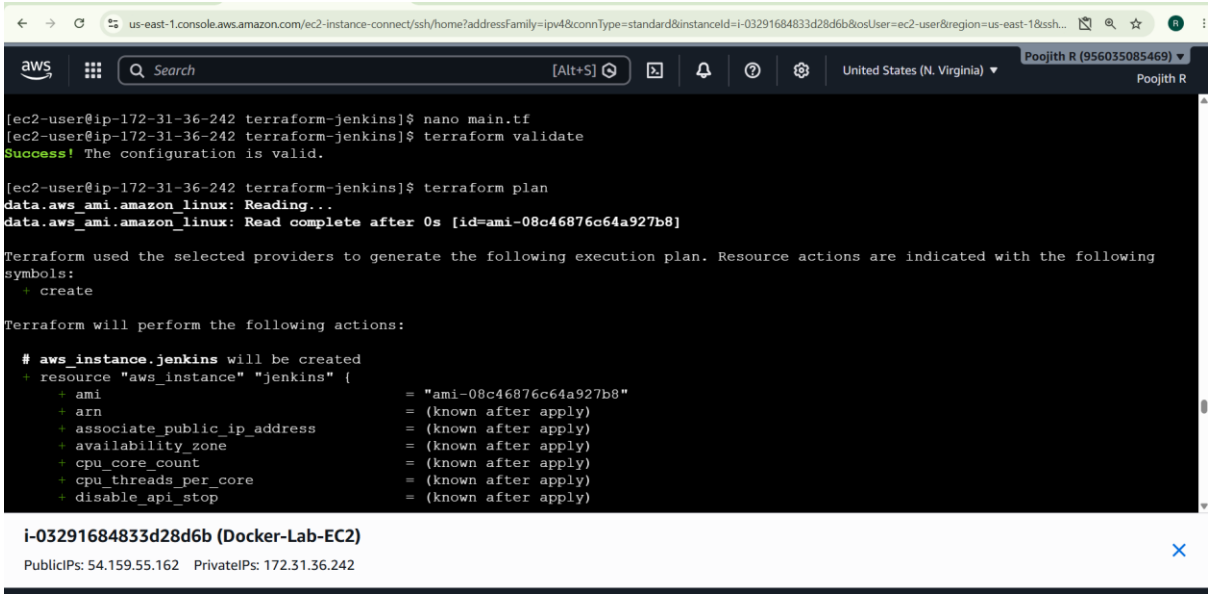

us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?addressFamily=ipv4&connType=standard&instanceId=i-03291684833d28d6b&osUser=ec2-user&region=us-east-1&ssh...
aws
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
[ec2-user@ip-172-31-36-242 terraform-jenkins]$ cat main.tf
terraform {
required_providers {
aws = {
source = "hashicorp/aws"
version = "~> 5.0"
}
}
}
provider "aws" {
region = "us-east-1"
}
data "aws_ami" "amazon_linux" {
most_recent = true
owners = ["amazon"]
filter {
name = "name"
values = ["al2023-ami-2023.*-x86_64"]
}
filter {
i-03291684833d28d6b (Docker-Lab-EC2)
PublicIPs: 54.159.55.162 PrivateIPs: 172.31.36.242

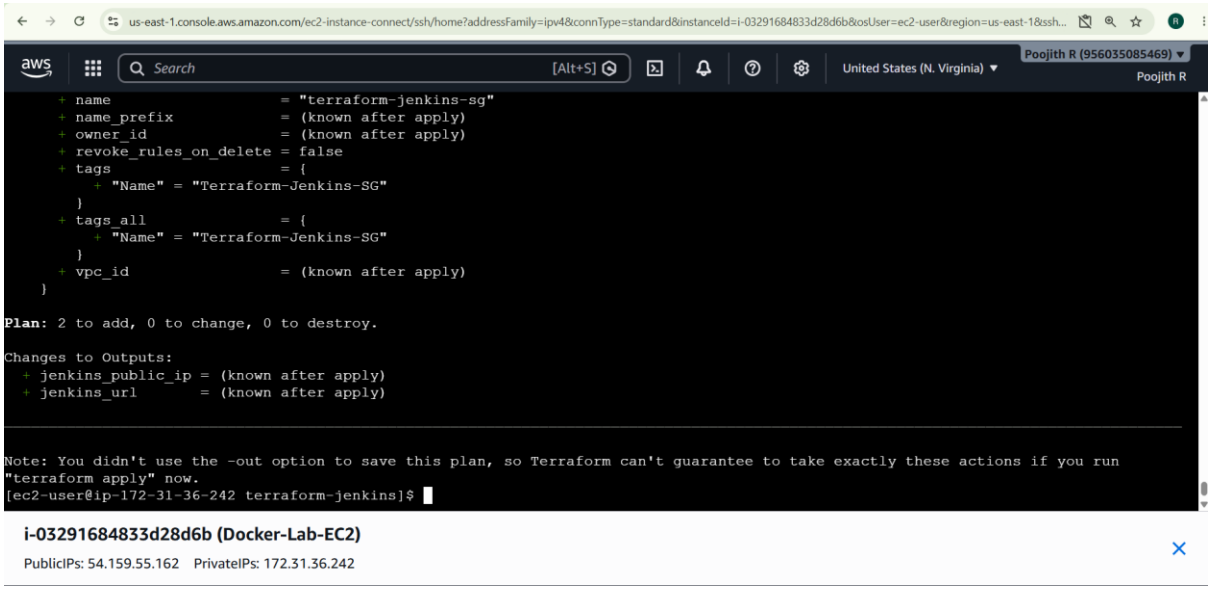

us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?addressFamily=ipv4&connType=standard&instanceId=i-03291684833d28d6b&osUser=ec2-user&region=us-east-1&ssh...
aws
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
}
resource "aws_security_group" "jenkins_sg" {
name = "terraform-jenkins-sg"
description = "Allow SSH and Jenkins web access"
ingress {
description = "SSH access"
from_port = 22
to_port = 22
protocol = "tcp"
cidr_blocks = ["0.0.0.0/0"]
}
ingress {
description = "Jenkins web access"
from_port = 8080
to_port = 8080
protocol = "tcp"
cidr_blocks = ["0.0.0.0/0"]
}
egress {
from_port = 0
i-03291684833d28d6b (Docker-Lab-EC2)
PublicIPs: 54.159.55.162 PrivateIPs: 172.31.36.242

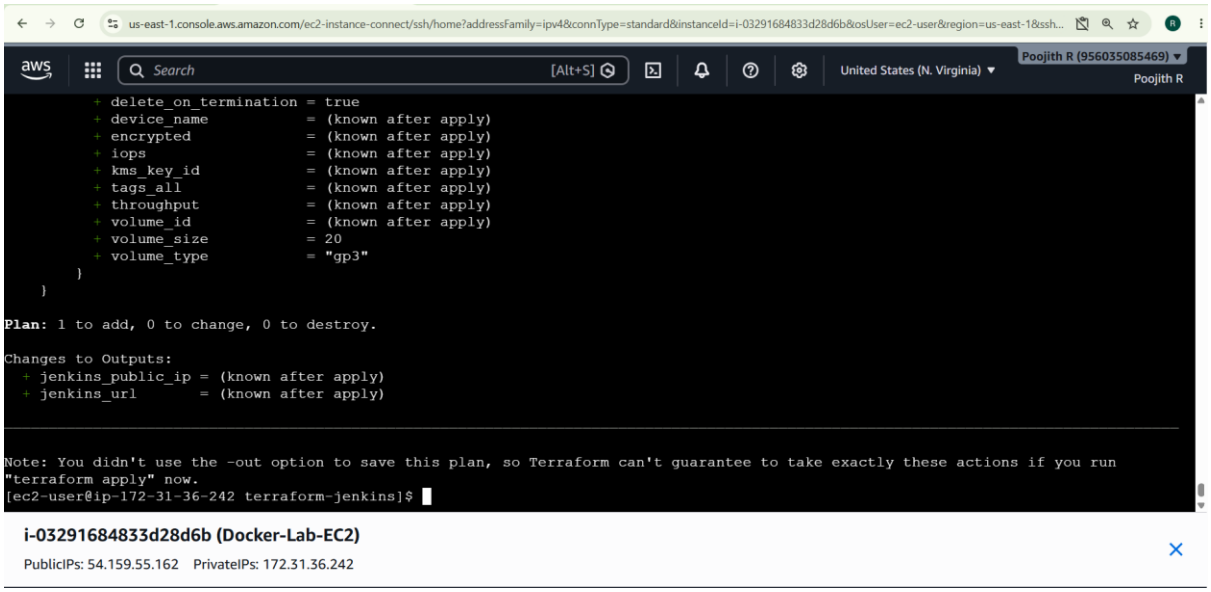

us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh?addressFamily=ipv4&connType=standard&instanceId=i-03291684833d28d6b&osUser=ec2-user&region=us-east-1&ssh...
aws
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
tags = {
Name = "Terraform-Jenkins-SG"
}
}
resource "aws_instance" "jenkins" {
ami = data.aws_ami.amazon_linux.id
instance_type = "t3.medium"
key_name = "jenkins-lab-key"
vpc_security_group_ids = [aws_security_group.jenkins_sg.id]
user_data = <<-USERDATA
#!/bin/bash
set -euxo pipefail
dnf update -y
dnf install -y fontconfig java-21-amazon-corretto wget
wget -O /etc/yum.repos.d/jenkins.repo \
https://pkg.jenkins.io/rpm-stable/jenkins.repo
rpm --import https://pkg.jenkins.io/rpm-stable/jenkins.io-2023.key
i-03291684833d28d6b (Docker-Lab-EC2)
PublicIPs: 54.159.55.162 PrivateIPs: 172.31.36.242

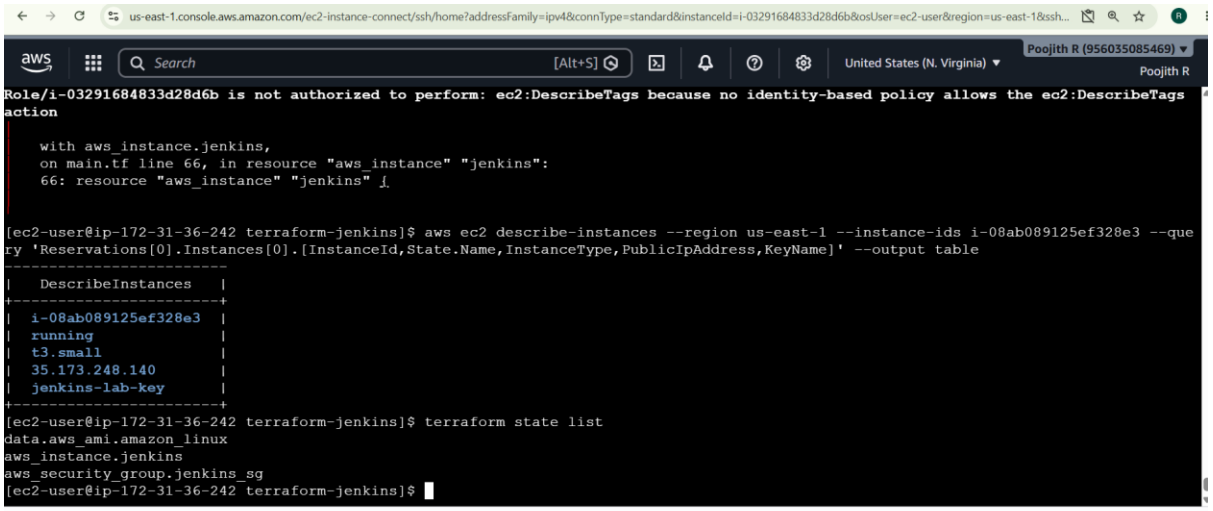

aws
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
systemctl enable jenkins
systemctl start jenkins
USERDATA
root_block_device {
volume_size = 20
volume_type = "gp3"
}
tags = {
Name = "Terraform-Jenkins-Server"
}
}
output "jenkins_public_ip" {
description = "Public IPv4 address of the Jenkins server"
value = aws_instance.jenkins.public_ip
}
output "jenkins_url" {
description = "Jenkins web URL"
value = "http://${aws_instance.jenkins.public_ip}:8080"
}
[ec2-user@ip-172-31-36-242 terraform-jenkins]$


us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh?addressFamily=ipv4&connType=standard&instanceId=i-03291684833d28d6b&osUser=ec2-user&region=us-east-1&ssh...
aws
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.
Terraform has been successfully initialized!
You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.
If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
[ec2-user@ip-172-31-36-242 terraform-jenkins]$ terraform validate
Success! The configuration is valid.
[ec2-user@ip-172-31-36-242 terraform-jenkins]$
i-03291684833d28d6b (Docker-Lab-EC2)
PublicIPs: 54.159.55.162 PrivateIPs: 172.31.36.242


us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?addressFamily=ipv4&connType=standard&instanceId=i-03291684833d28d6b&osUser=ec2-user&region=us-east-1&ssh...
aws
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
[ec2-user@ip-172-31-36-242 terraform-jenkins]$ nano main.tf
[ec2-user@ip-172-31-36-242 terraform-jenkins]$ terraform validate
Success! The configuration is valid.
[ec2-user@ip-172-31-36-242 terraform-jenkins]$ terraform plan
data.aws_ami.amazon_linux: Reading...
data.aws_ami.amazon_linux: Read complete after 0s [id=ami-08c46876c64a927b8]
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create
Terraform will perform the following actions:
aws_instance.jenkins will be created
+ resource "aws_instance" "jenkins" {
+ ami = "ami-08c46876c64a927b8"
+ arn = (known after apply)
+ associate_public_ip_address = (known after apply)
+ availability_zone = (known after apply)
+ cpu_core_count = (known after apply)
+ cpu_threads_per_core = (known after apply)
+ disable_api_stop = (known after apply)
i-03291684833d28d6b (Docker-Lab-EC2)
PublicIPs: 54.159.55.162 PrivateIPs: 172.31.36.242

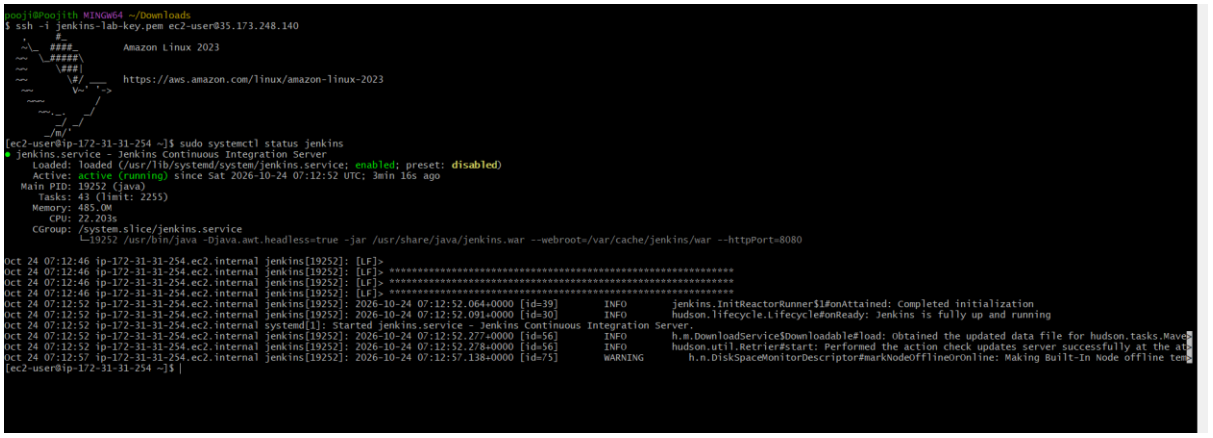

us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?addressFamily=ipv4&connType=standard&instanceId=i-03291684833d28d6b&osUser=ec2-user&region=us-east-1&ssh...
aws
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
+ name = "terraform-jenkins-sg"
+ name_prefix = (known after apply)
+ owner_id = (known after apply)
+ revoke_rules_on_delete = false
+ tags = {
+ "Name" = "Terraform-Jenkins-SG"
}
+ tags_all = {
+ "Name" = "Terraform-Jenkins-SG"
}
+ vpc_id = (known after apply)
}
Plan: 2 to add, 0 to change, 0 to destroy.
Changes to Outputs:
+ jenkins_public_ip = (known after apply)
+ jenkins_url = (known after apply)
Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
[ec2-user@ip-172-31-36-242 terraform-jenkins]$
i-03291684833d28d6b (Docker-Lab-EC2)
PublicIPs: 54.159.55.162 PrivateIPs: 172.31.36.242


us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?addressFamily=ipv4&connType=standard&instanceId=i-03291684833d28d6b&osUser=ec2-user&region=us-east-1&ssh...
aws
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
+ delete_on_termination = true
+ device_name = (known after apply)
+ encrypted = (known after apply)
+ iops = (known after apply)
+ kms_key_id = (known after apply)
+ tags_all = (known after apply)
+ throughput = (known after apply)
+ volume_id = (known after apply)
+ volume_size = 20
+ volume_type = "gp3"
}
}
Plan: 1 to add, 0 to change, 0 to destroy.
Changes to Outputs:
+ jenkins_public_ip = (known after apply)
+ jenkins_url = (known after apply)
Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
[ec2-user@ip-172-31-36-242 terraform-jenkins]$
i-03291684833d28d6b (Docker-Lab-EC2)
PublicIPs: 54.159.55.162 PrivateIPs: 172.31.36.242

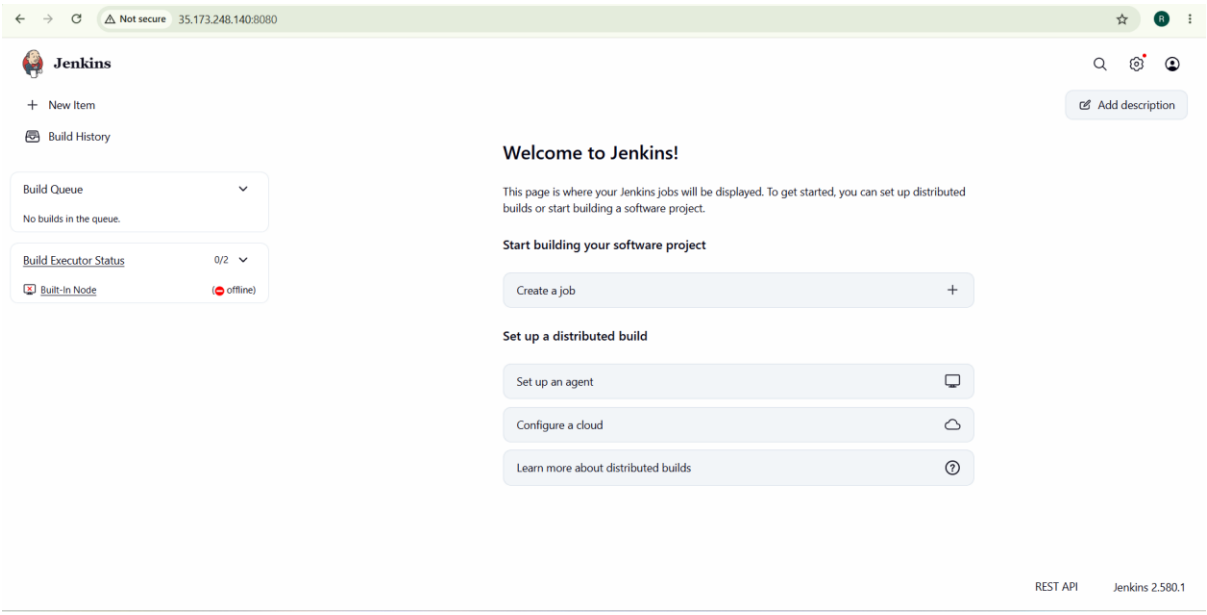


i-03291684833d28d6b (Docker-Lab-EC2)

PublicIPs: 54.159.55.162 PrivateIPs: 172.31.36.242







5. Create one Jenkins job to build and push the Docker image to Docker Hub.
    - Source: https://github.com/betawins/Python-app.git

    Source Codes: https://github.com/betawins/docker-tasks.git

Objective: Automate the process of cloning the Python application source code, building its Docker image, and pushing the image to Docker Hub.

Source repository: Python-app

Implementation steps

1. Opened the Jenkins dashboard.
2. Installed or verified the required Git and Pipeline plugins.
3. Verified that Git and Docker were available on the Jenkins execution environment.
4. Created the Docker Hub repository named pythonapp under the account poojith7878.
5. Opened Manage Jenkins → Credentials and added the Docker Hub credentials.
6. Configured the credential ID as dockerhub-creds, using the Docker Hub username and personal access token.

7. Created a new Jenkins Pipeline job named Python-App-Docker-Pipeline.
8. Configured the pipeline to retrieve source code from the Python application repository.
9. Added the pipeline script to the job configuration.

```
pipeline {
  agent any

  stages {
    stage('Checkout') {
      steps {
        git branch: 'main',
          url: 'https://github.com/betawins/Python-app.git'
      }
    }

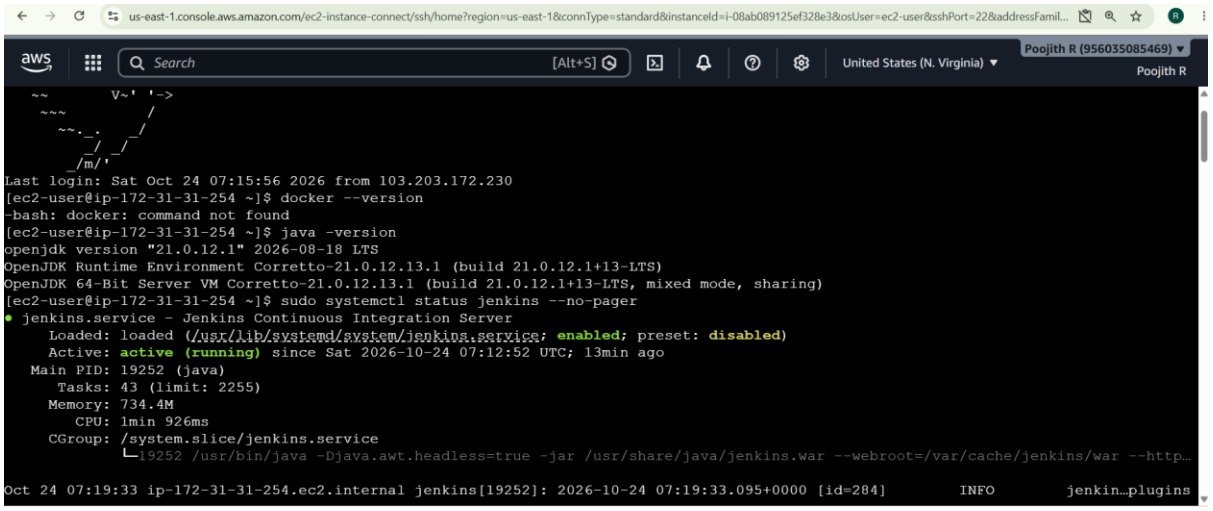
    stage('Build Docker Image') {
      steps {
        sh 'docker build -t poojith7878/pythonapp:${BUILD_NUMBER} .'
      }
    }

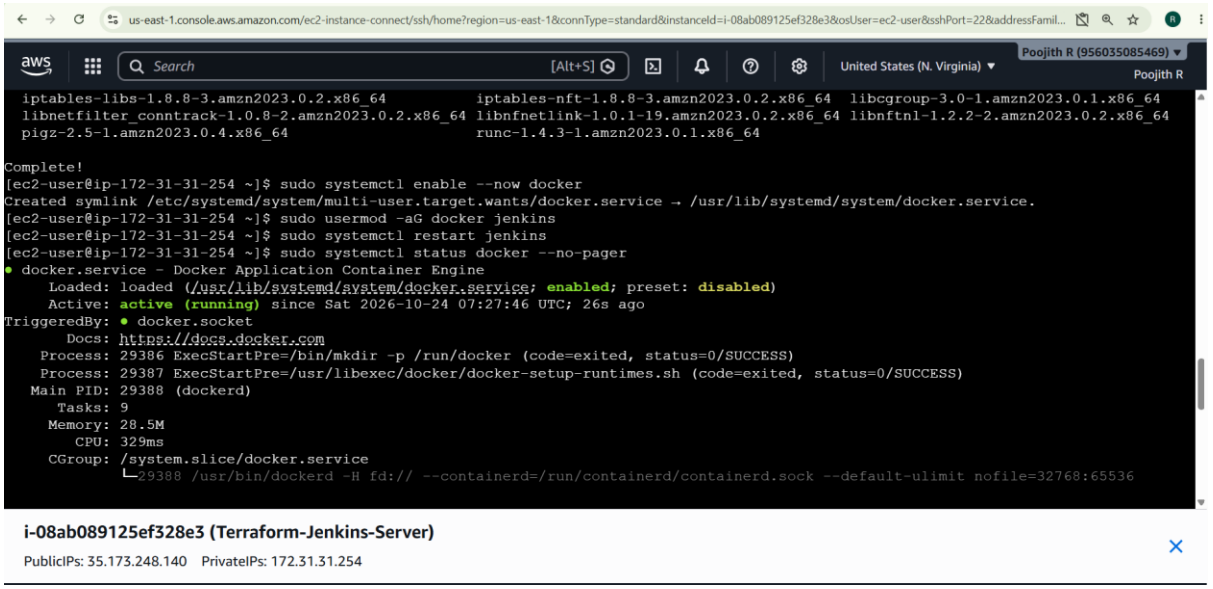
    stage('Login and Push to Docker Hub') {
      steps {
        withCredentials([usernamePassword(
          credentialsId: 'dockerhub-creds',
          usernameVariable: 'DOCKERHUB_USER',
          passwordVariable: 'DOCKERHUB_TOKEN'
        )]) {
```

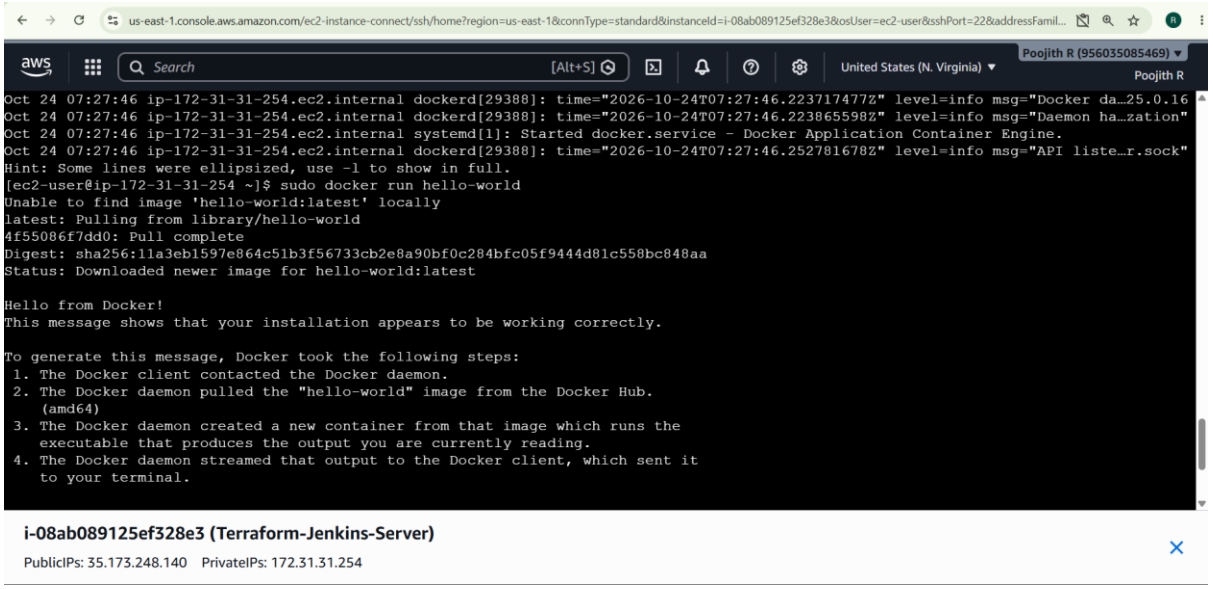
```
                sh '''
                  echo "$DOCKERHUB_TOKEN" | docker login \
                    -u "$DOCKERHUB_USER" --password-stdin
                  docker push poojith7878/pythonapp:${BUILD_NUMBER}
                '''
              }
            }
          }
        }

        post {
          always {
            sh 'docker logout || true'
          }
        }
}
```

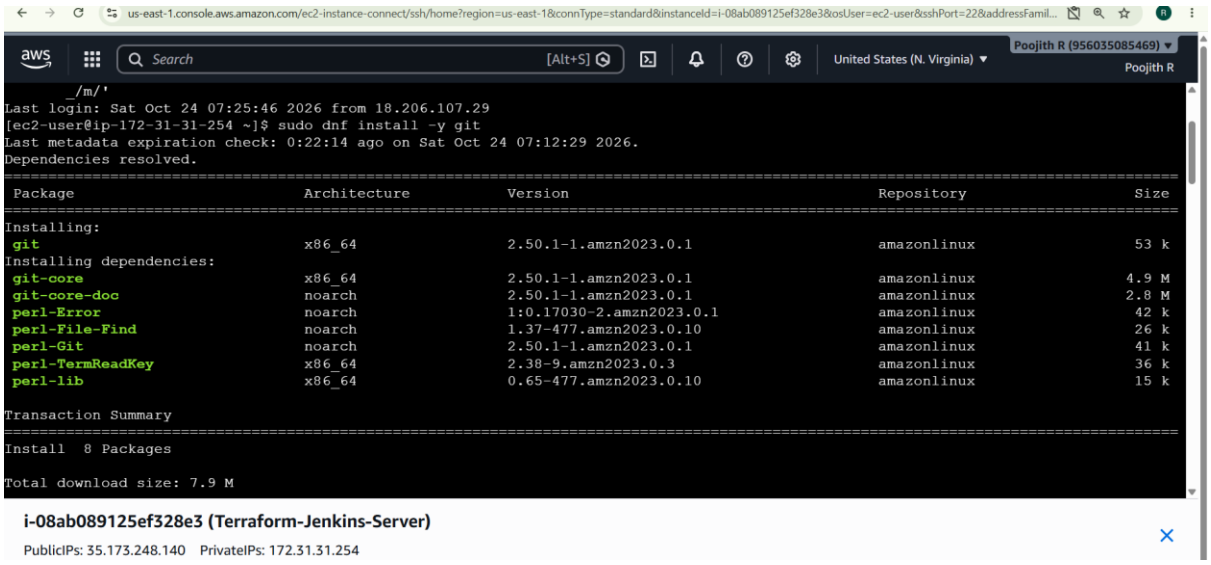
10. Saved the pipeline configuration.
11. Selected Build Now to execute the job.
12. Opened Console Output and verified that Git checkout, Docker image build, Docker Hub login, and image push completed successfully.
13. Confirmed that the first successful build pushed the 1 tag to Docker Hub.

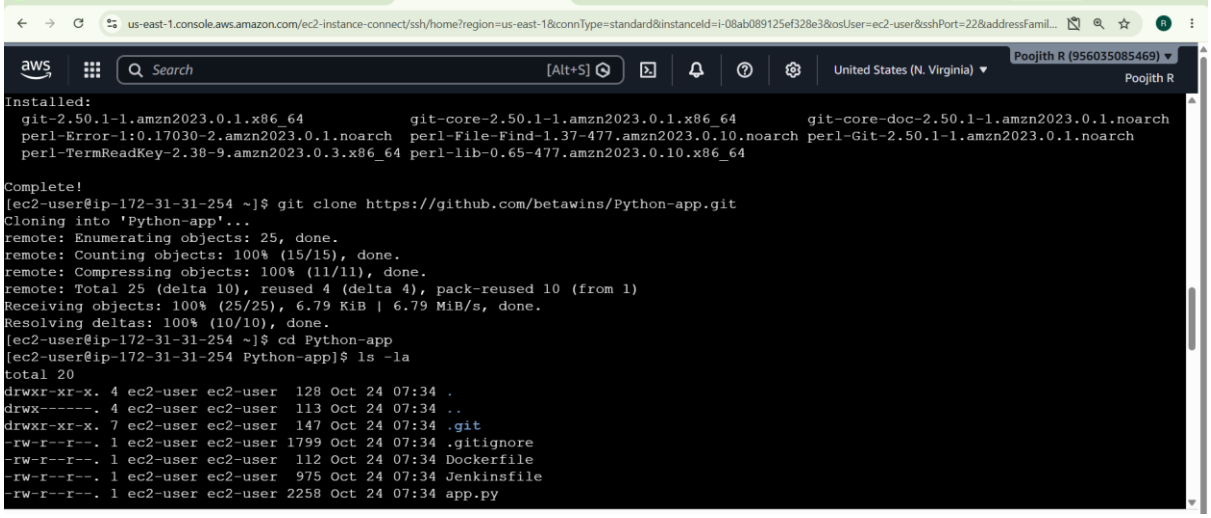

us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=us-east-1&connType=standard&instanceId=i-08ab089125ef328e3&osUser=ec2-user&sshPort=22&addressFamil...
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
Last login: Sat Oct 24 07:15:56 2026 from 103.203.172.230
[ec2-user@ip-172-31-31-254 ~]$ docker --version
-bash: docker: command not found
[ec2-user@ip-172-31-31-254 ~]$ java -version
openjdk version "21.0.12.1" 2026-08-18 LTS
OpenJDK Runtime Environment Corretto-21.0.12.13.1 (build 21.0.12.1+13-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.13.1 (build 21.0.12.1+13-LTS, mixed mode, sharing)
[ec2-user@ip-172-31-31-254 ~]$ sudo systemctl status jenkins --no-pager
● jenkins.service - Jenkins Continuous Integration Server
Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)
Active: active (running) since Sat 2026-10-24 07:12:52 UTC; 13min ago
Main PID: 19252 (java)
Tasks: 43 (limit: 2255)
Memory: 734.4M
CPU: 1min 926ms
CGroup: /system.slice/jenkins.service
└19252 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --http…
Oct 24 07:19:33 ip-172-31-31-254.ec2.internal jenkins[19252]: 2026-10-24 07:19:33.095+0000 [id=284] INFO jenkin…plugins
i-08ab089125ef328e3 (Terraform-Jenkins-Server)
PublicIPs: 35.173.248.140 PrivateIPs: 172.31.31.254

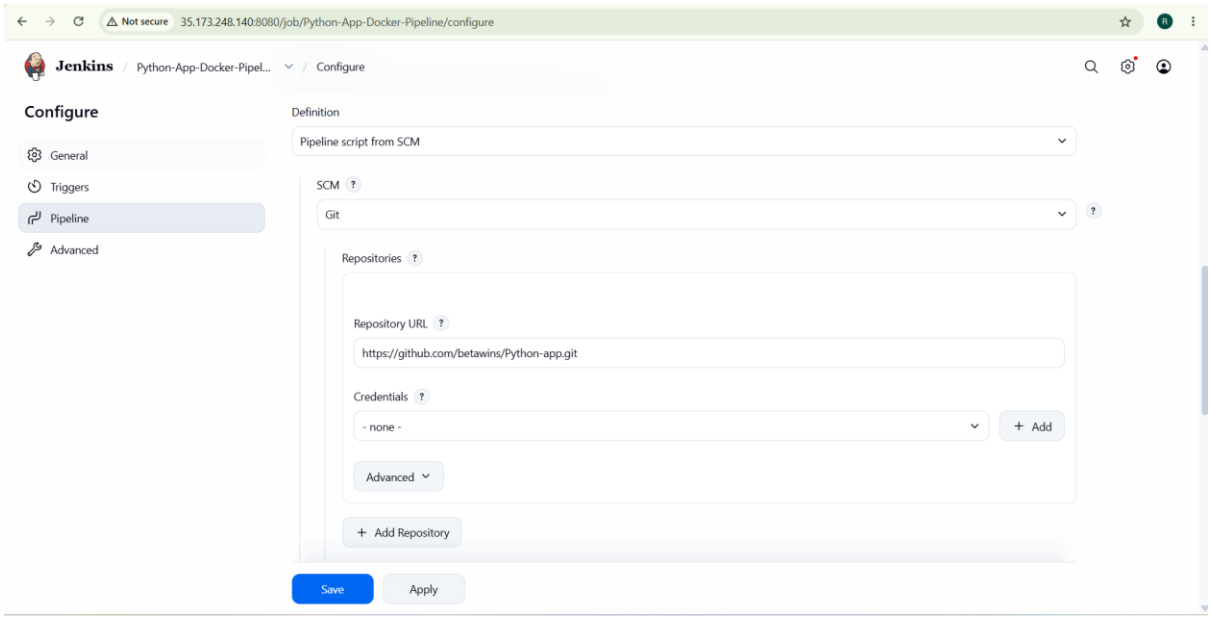

us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=us-east-1&connType=standard&instanceId=i-08ab089125ef328e3&osUser=ec2-user&sshPort=22&addressFamil...
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
iptables-libs-1.8.8-3.amzn2023.0.2.x86_64 iptables-nft-1.8.8-3.amzn2023.0.2.x86_64 libcgroup-3.0-1.amzn2023.0.1.x86_64
libnetfilter_conntrack-1.0.8-2.amzn2023.0.2.x86_64 libnfnetlink-1.0.1-19.amzn2023.0.2.x86_64 libnftnl-1.2.2-2.amzn2023.0.2.x86_64
pigz-2.5-1.amzn2023.0.4.x86_64 runc-1.4.3-1.amzn2023.0.1.x86_64
Complete!
[ec2-user@ip-172-31-31-254 ~]$ sudo systemctl enable --now docker
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service → /usr/lib/systemd/system/docker.service.
[ec2-user@ip-172-31-31-254 ~]$ sudo usermod -aG docker jenkins
[ec2-user@ip-172-31-31-254 ~]$ sudo systemctl restart jenkins
[ec2-user@ip-172-31-31-254 ~]$ sudo systemctl status docker --no-pager
● docker.service - Docker Application Container Engine
Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: disabled)
Active: active (running) since Sat 2026-10-24 07:27:46 UTC; 26s ago
TriggeredBy: ● docker.socket
Docs: https://docs.docker.com
Process: 29386 ExecStartPre=/bin/mkdir -p /run/docker (code=exited, status=0/SUCCESS)
Process: 29387 ExecStartPre=/usr/libexec/docker/docker-setup-runtimes.sh (code=exited, status=0/SUCCESS)
Main PID: 29388 (dockerd)
Tasks: 9
Memory: 28.5M
CPU: 329ms
CGroup: /system.slice/docker.service
└29388 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock --default-ulimit nofile=32768:65536
i-08ab089125ef328e3 (Terraform-Jenkins-Server)
PublicIPs: 35.173.248.140 PrivateIPs: 172.31.31.254

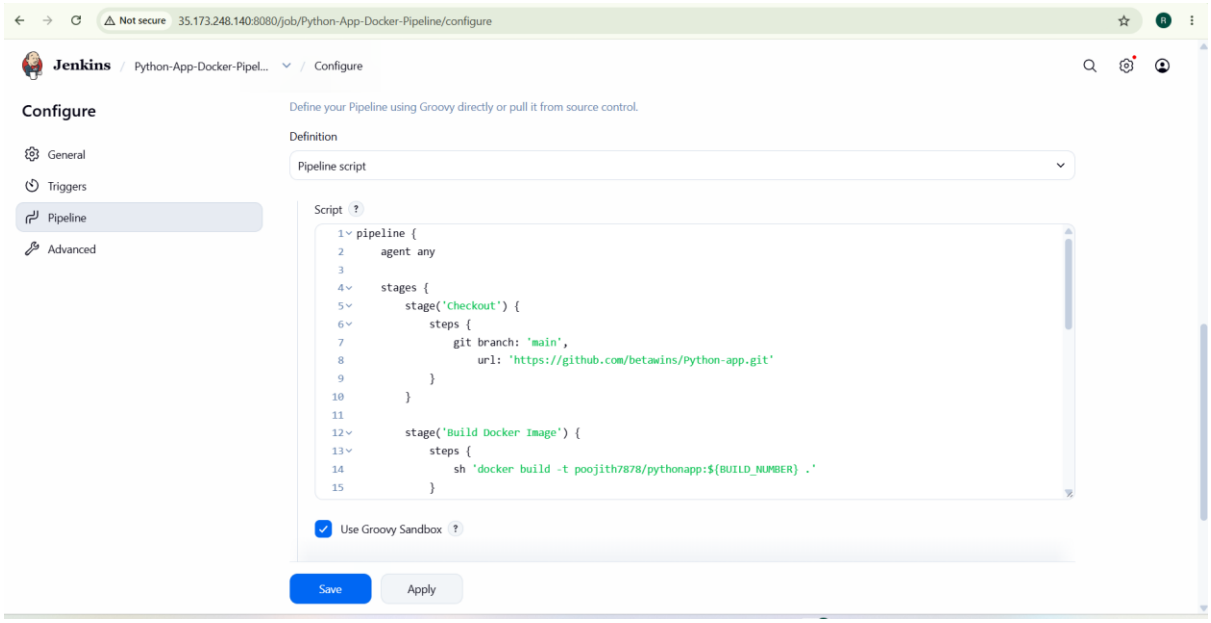

us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=us-east-1&connType=standard&instanceId=i-08ab089125ef328e3&osUser=ec2-user&sshPort=22&addressFamil...
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
Oct 24 07:27:46 ip-172-31-31-254.ec2.internal dockerd[29388]: time="2026-10-24T07:27:46.223717477Z" level=info msg="Docker da…25.0.16
Oct 24 07:27:46 ip-172-31-31-254.ec2.internal dockerd[29388]: time="2026-10-24T07:27:46.223865598Z" level=info msg="Daemon ha…zation"
Oct 24 07:27:46 ip-172-31-31-254.ec2.internal systemd[1]: Started docker.service - Docker Application Container Engine.
Oct 24 07:27:46 ip-172-31-31-254.ec2.internal dockerd[29388]: time="2026-10-24T07:27:46.252781678Z" level=info msg="API liste…r.sock"
Hint: Some lines were ellipsized, use -l to show in full.
[ec2-user@ip-172-31-31-254 ~]$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
Digest: sha256:11a3eb1597e864c51b3f56733cb2e8a90bf0c284bfc05f9444d81c558bc848aa
Status: Downloaded newer image for hello-world:latest
Hello from Docker!
This message shows that your installation appears to be working correctly.
To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
(amd64)
3. The Docker daemon created a new container from that image which runs the
executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
to your terminal.
i-08ab089125ef328e3 (Terraform-Jenkins-Server)
PublicIPs: 35.173.248.140 PrivateIPs: 172.31.31.254

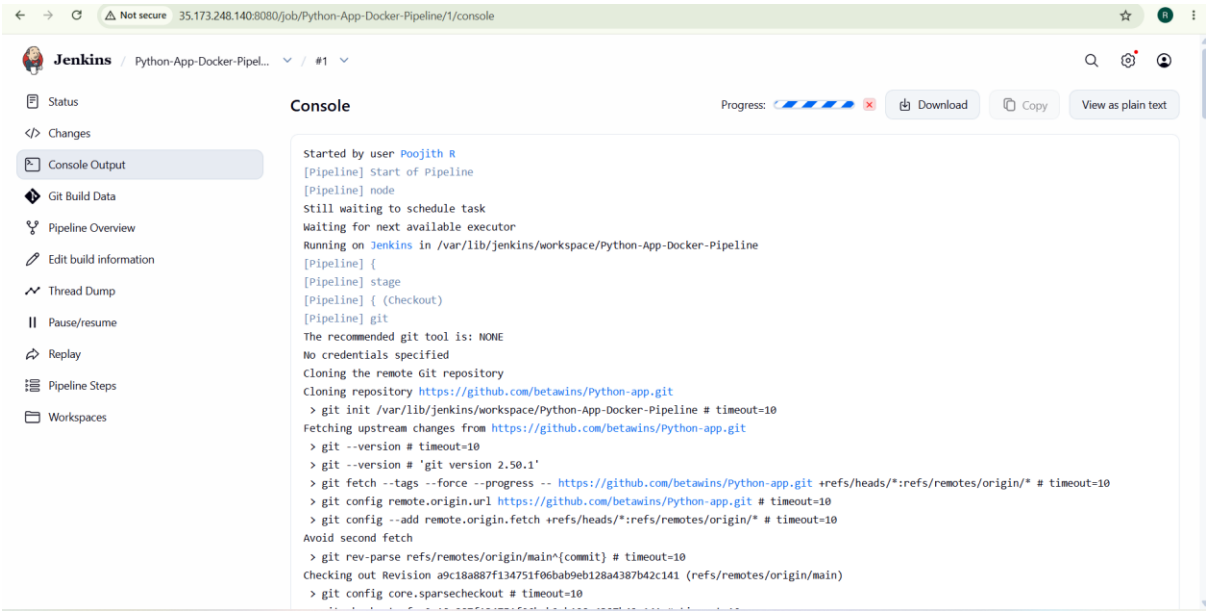

Not secure 35.173.248.140:8080/manage/credentials/store/system/domain/_/
Jenkins / Manage Jenkins / Credentials / System / Global
Global
Credentials that should be available everywhere.
+ Add Credentials
dockerhub-creds poojith7878/******
Docker Hub credentials for image push

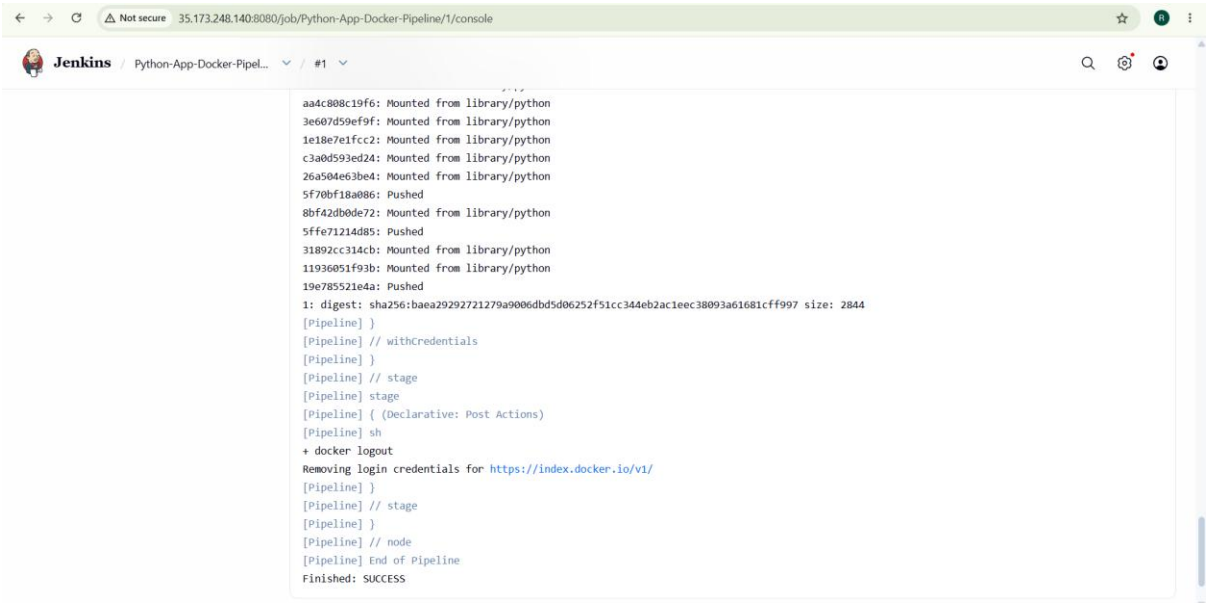

us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=us-east-1&connType=standard&instanceId=i-08ab089125ef328e3&osUser=ec2-user&sshPort=22&addressFamil...
aws
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
_/m/'
Last login: Sat Oct 24 07:25:46 2026 from 18.206.107.29
[ec2-user@ip-172-31-31-254 ~]$ sudo dnf install -y git
Last metadata expiration check: 0:22:14 ago on Sat Oct 24 07:12:29 2026.
Dependencies resolved.
Package Architecture Version Repository Size
Installing:
git x86_64 2.50.1-1.amzn2023.0.1 amazonlinux 53 k
Installing dependencies:
git-core x86_64 2.50.1-1.amzn2023.0.1 amazonlinux 4.9 M
git-core-doc noarch 2.50.1-1.amzn2023.0.1 amazonlinux 2.8 M
perl-Error noarch 1:0.17030-2.amzn2023.0.1 amazonlinux 42 k
perl-File-Find noarch 1.37-477.amzn2023.0.10 amazonlinux 26 k
perl-Git noarch 2.50.1-1.amzn2023.0.1 amazonlinux 41 k
perl-TermReadKey x86_64 2.38-9.amzn2023.0.3 amazonlinux 36 k
perl-lib x86_64 0.65-477.amzn2023.0.10 amazonlinux 15 k
Transaction Summary
Install 8 Packages
Total download size: 7.9 M
i-08ab089125ef328e3 (Terraform-Jenkins-Server)
PublicIPs: 35.173.248.140 PrivateIPs: 172.31.31.254

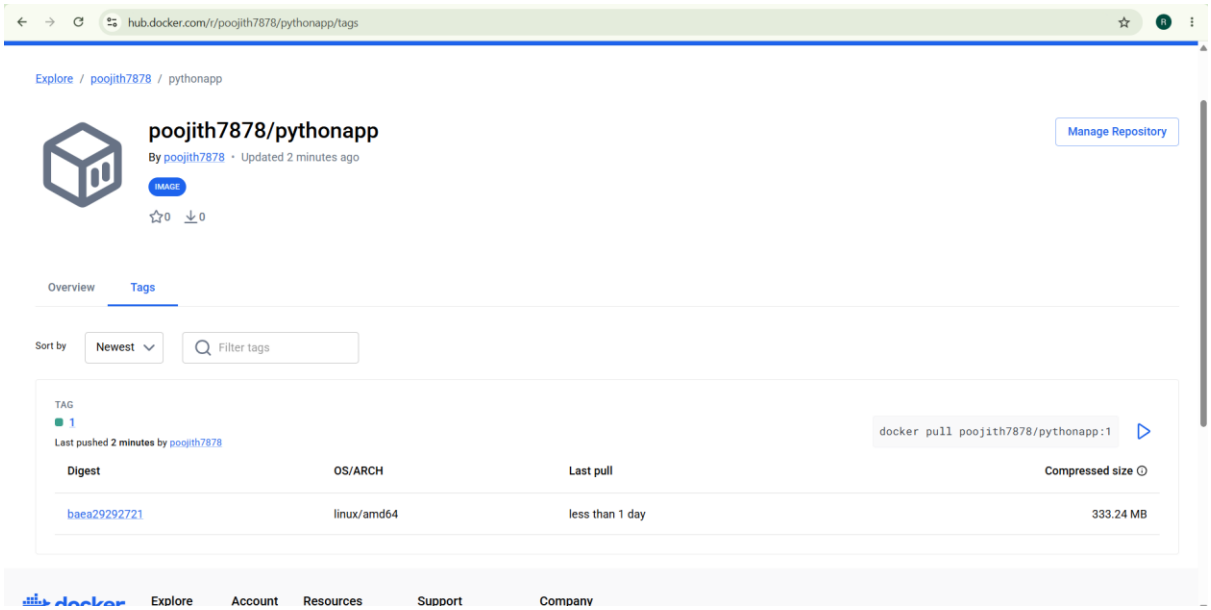

us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=us-east-1&connType=standard&instanceId=i-08ab089125ef328e3&osUser=ec2-user&sshPort=22&addressFamil...
aws
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
Installed:
git-2.50.1-1.amzn2023.0.1.x86_64 git-core-2.50.1-1.amzn2023.0.1.x86_64 git-core-doc-2.50.1-1.amzn2023.0.1.noarch
perl-Error-1:0.17030-2.amzn2023.0.1.noarch perl-File-Find-1.37-477.amzn2023.0.10.noarch perl-Git-2.50.1-1.amzn2023.0.1.noarch
perl-TermReadKey-2.38-9.amzn2023.0.3.x86_64 perl-lib-0.65-477.amzn2023.0.10.x86_64
Complete!
[ec2-user@ip-172-31-31-254 ~]$ git clone https://github.com/betawins/Python-app.git
Cloning into 'Python-app'...
remote: Enumerating objects: 25, done.
remote: Counting objects: 100% (15/15), done.
remote: Compressing objects: 100% (11/11), done.
remote: Total 25 (delta 10), reused 4 (delta 4), pack-reused 10 (from 1)
Receiving objects: 100% (25/25), 6.79 KiB | 6.79 MiB/s, done.
Resolving deltas: 100% (10/10), done.
[ec2-user@ip-172-31-31-254 ~]$ cd Python-app
[ec2-user@ip-172-31-31-254 Python-app]$ ls -la
total 20
drwxr-xr-x. 4 ec2-user ec2-user 128 Oct 24 07:34 .
drwx------. 4 ec2-user ec2-user 113 Oct 24 07:34 ..
drwxr-xr-x. 7 ec2-user ec2-user 147 Oct 24 07:34 .git
-rw-r--r--. 1 ec2-user ec2-user 1799 Oct 24 07:34 .gitignore
-rw-r--r--. 1 ec2-user ec2-user 112 Oct 24 07:34 Dockerfile
-rw-r--r--. 1 ec2-user ec2-user 975 Oct 24 07:34 Jenkinsfile
-rw-r--r--. 1 ec2-user ec2-user 2258 Oct 24 07:34 app.py


Not secure 35.173.248.140:8080/job/Python-App-Docker-Pipeline/configure
Jenkins / Python-App-Docker-Pipel... / Configure
Configure
General
Triggers
Pipeline
Advanced
Definition
Pipeline script from SCM
SCM
Git
Repositories
Repository URL
https://github.com/betawins/Python-app.git
Credentials
- none -
+ Add
Advanced
+ Add Repository
Save
Apply


Not secure 35.173.248.140:8080/job/Python-App-Docker-Pipeline/configure
Jenkins / Python-App-Docker-Pipel... / Configure
Configure
General
Triggers
Pipeline
Advanced
Define your Pipeline using Groovy directly or pull it from source control.
Definition
Pipeline script
Script
pipeline {
agent any
stages {
stage('Checkout') {
steps {
git branch: 'main',
url: 'https://github.com/betawins/Python-app.git'
}
}
stage('Build Docker Image') {
steps {
sh 'docker build -t poojith7878/pythonapp:${BUILD_NUMBER} .'
}
Use Groovy Sandbox
Save
Apply


Not secure 35.173.248.140:8080/job/Python-App-Docker-Pipeline/1/console
Jenkins / Python-App-Docker-Pipel... / #1
Status
Changes
Console Output
Git Build Data
Pipeline Overview
Edit build information
Thread Dump
Pause/resume
Replay
Pipeline Steps
Workspaces
Console
Progress:
Download
Copy
View as plain text
Started by user Poojith R
[Pipeline] Start of Pipeline
[Pipeline] node
Still waiting to schedule task
Waiting for next available executor
Running on Jenkins in /var/lib/jenkins/workspace/Python-App-Docker-Pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout)
[Pipeline] git
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/betawins/Python-app.git
> git init /var/lib/jenkins/workspace/Python-App-Docker-Pipeline # timeout=10
Fetching upstream changes from https://github.com/betawins/Python-app.git
> git --version # timeout=10
> git --version # 'git version 2.50.1'
> git fetch --tags --force --progress -- https://github.com/betawins/Python-app.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git config remote.origin.url https://github.com/betawins/Python-app.git # timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision a9c18a887f134751f06bab9eb128a4387b42c141 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10


Not secure 35.173.248.140:8080/job/Python-App-Docker-Pipeline/1/console
Jenkins / Python-App-Docker-Pipel... / #1
aa4c808c19f6: Mounted from library/python
3e607d59ef9f: Mounted from library/python
1e18e7e1fcc2: Mounted from library/python
c3a0d593ed24: Mounted from library/python
26a504e63be4: Mounted from library/python
5f70bf18a086: Pushed
8bf42db0de72: Mounted from library/python
5ffe71214d85: Pushed
31892cc314cb: Mounted from library/python
11936051f93b: Mounted from library/python
19e785521e4a: Pushed
1: digest: sha256:baea29292721279a9006dbd5d06252f51cc344eb2ac1eec38093a61681cff997 size: 2844
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] sh
+ docker logout
Removing login credentials for https://index.docker.io/v1/
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS



- From the **frontend source code**, write a Dockerfile, build a Docker image, run it, and push that image to your Docker registry.

Objective: Containerize the static frontend application using Nginx, run it, and upload the image to Docker Hub.

Source repository: [docker-tasks]

Implementation steps

1. Connected to the Jenkins EC2 instance.
2. Cloned the source repository if it was not already available.

git clone https://github.com/betawins/docker-tasks.git

3. Navigated to the frontend source directory.

cd ~/docker-tasks/Frontend_based_source

4. Reviewed the application files, including index.html, javascript.js, style.css, and todayDeal.js.
5. Created a Dockerfile with the following configuration.

   FROM nginx:stable-alpine

   COPY . /usr/share/nginx/html/

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]

6. Built the frontend Docker image.

sudo docker build -t frontend-app:v1 .

7. Verified that the image was created.

sudo docker images

8. Started the container by mapping host port 8086 to container port 80.

sudo docker run -d --name frontend-app -p 8086:80 frontend-app:v1

In the completed lab, the container was named frontend-app and was tested on port 8086.

9. Verified that the application was responding.

curl -I http://localhost:8086

The response included HTTP 200 OK.

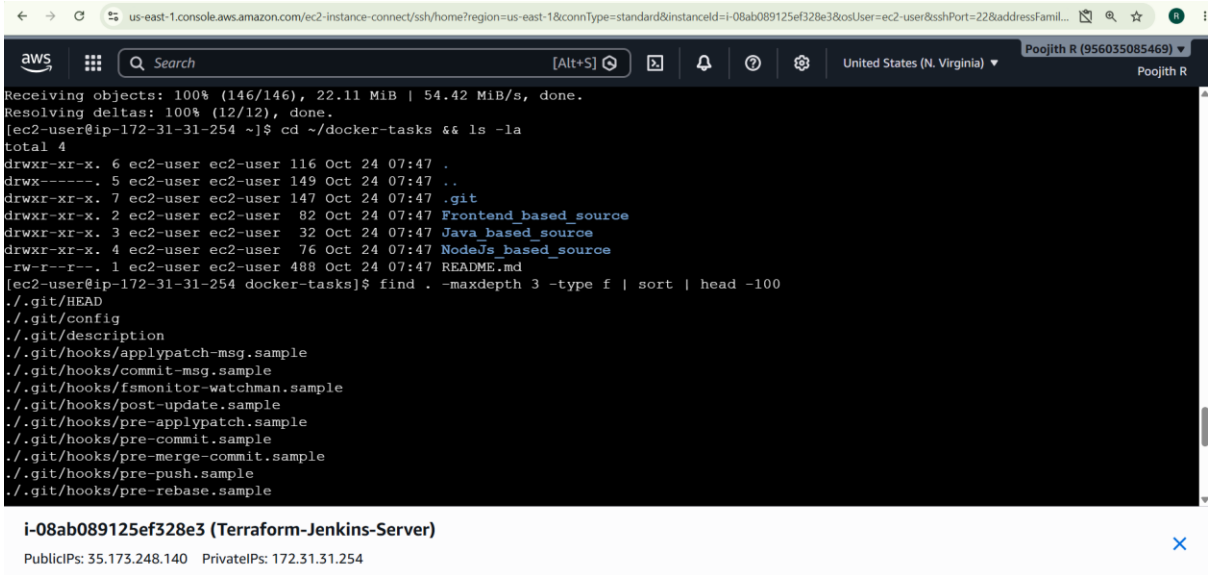
10. Tagged the image for Docker Hub.

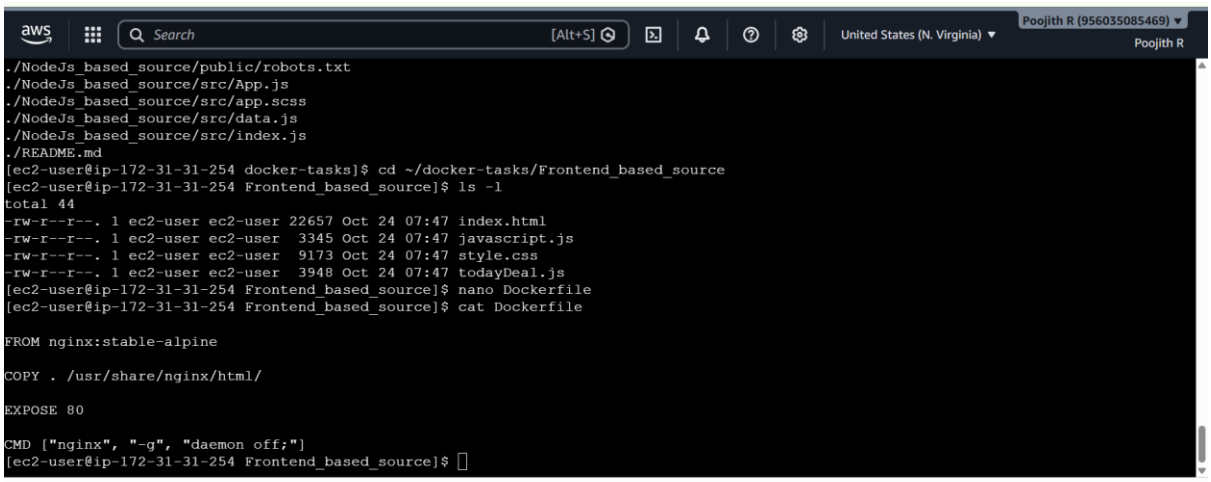
sudo docker tag frontend-app:v1 poojith7878/frontend-app:v1

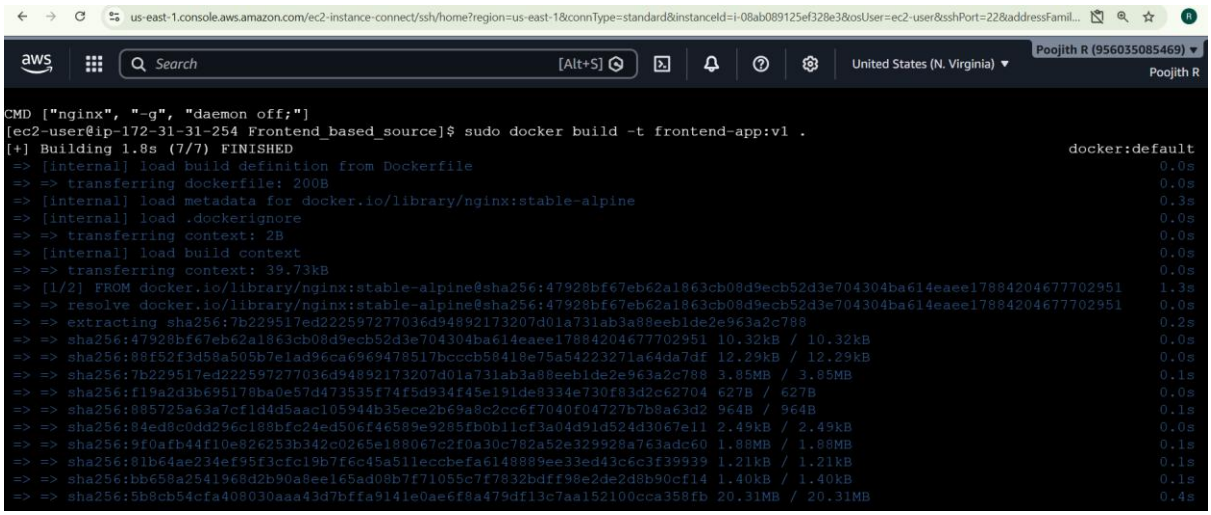
11. Pushed the image to Docker Hub.

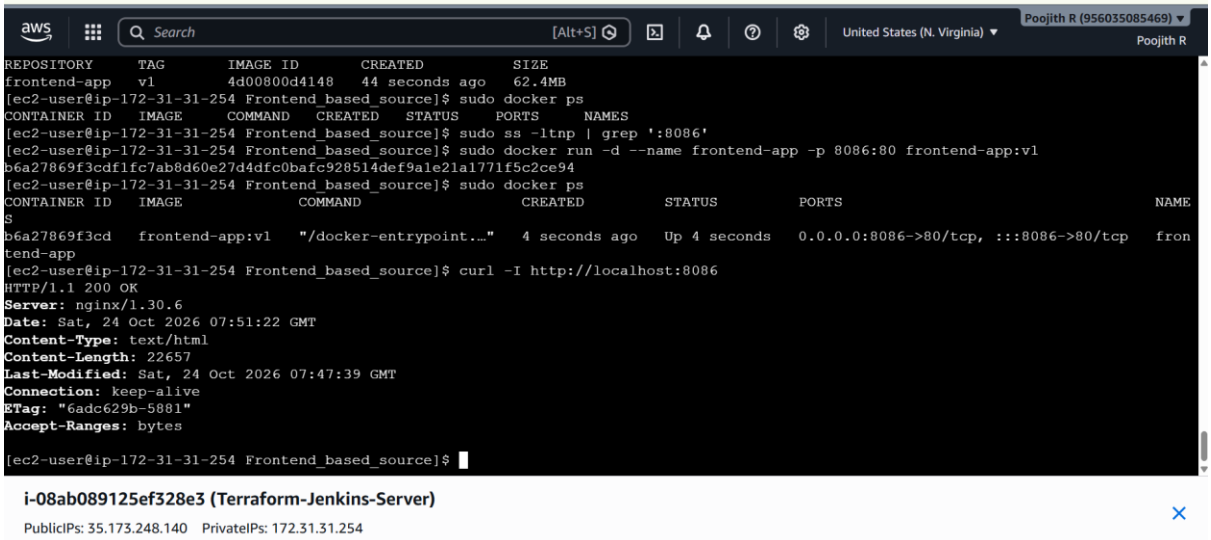
sudo docker push poojith7878/frontend-app:v1

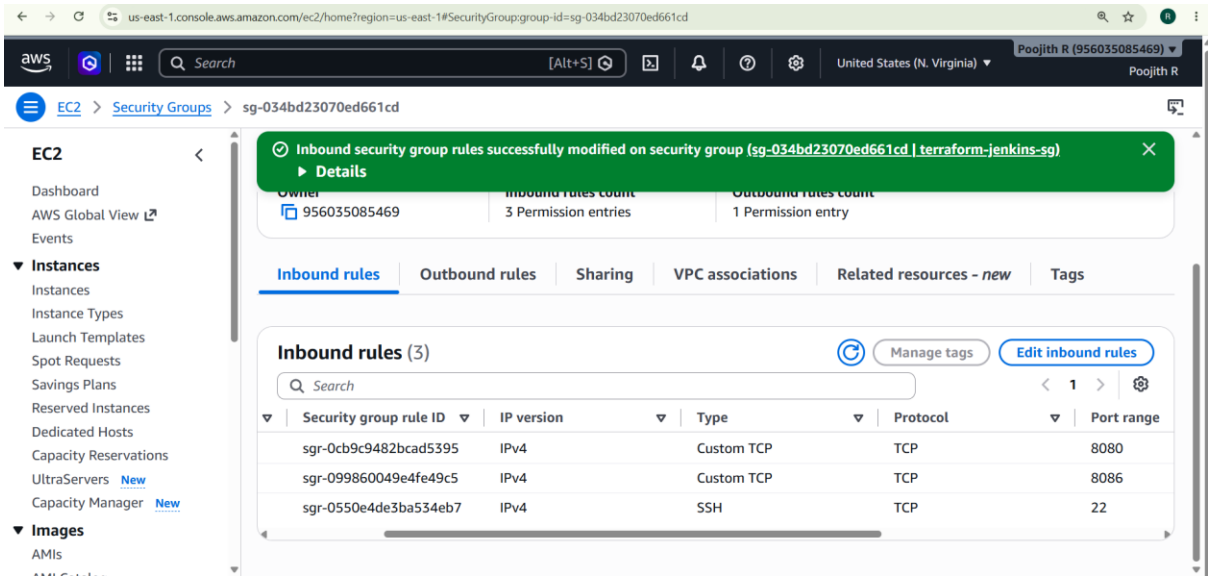
12. Opened Docker Hub and confirmed that the v1 tag was available.

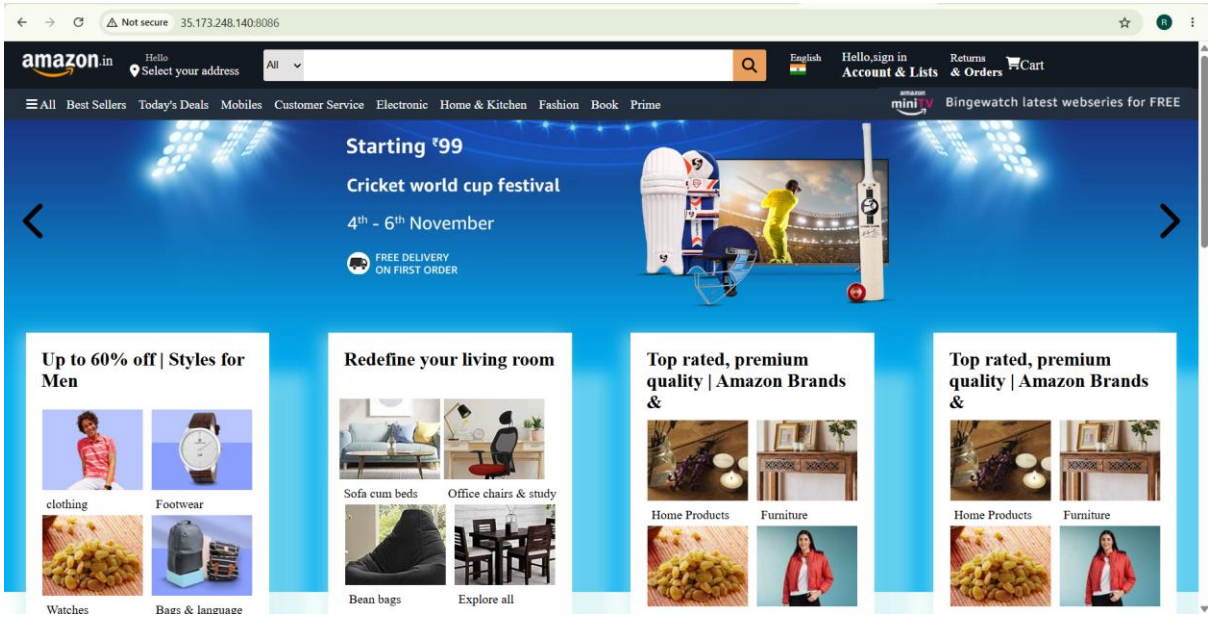


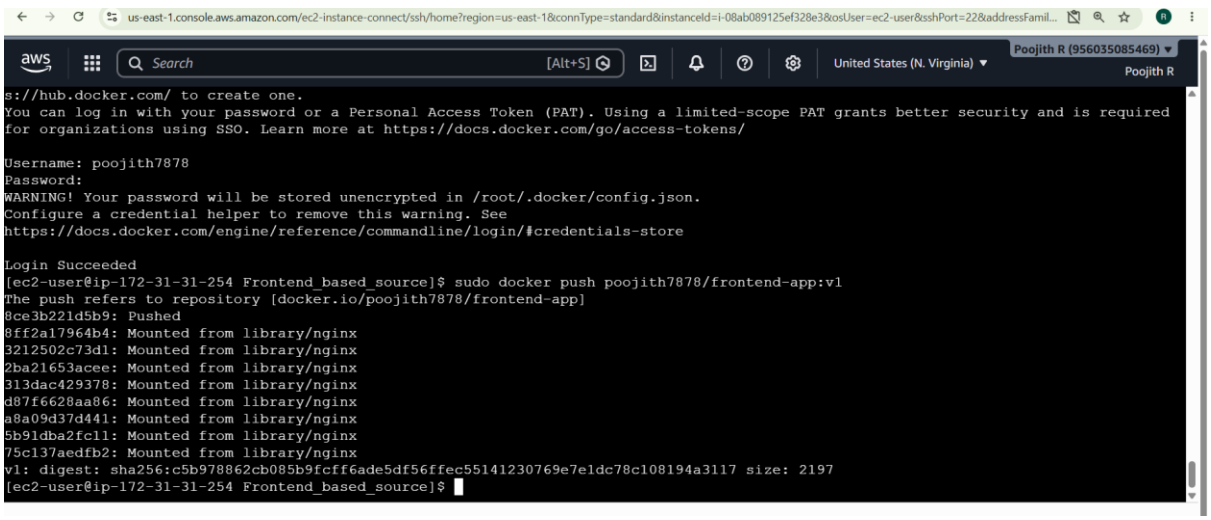

Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
./NodeJs_based_source/public/robots.txt
./NodeJs_based_source/src/App.js
./NodeJs_based_source/src/app.scss
./NodeJs_based_source/src/data.js
./NodeJs_based_source/src/index.js
./README.md
[ec2-user@ip-172-31-31-254 docker-tasks]$ cd ~/docker-tasks/Frontend_based_source
[ec2-user@ip-172-31-31-254 Frontend_based_source]$ ls -l
total 44
-rw-r--r--. 1 ec2-user ec2-user 22657 Oct 24 07:47 index.html
-rw-r--r--. 1 ec2-user ec2-user 3345 Oct 24 07:47 javascript.js
-rw-r--r--. 1 ec2-user ec2-user 9173 Oct 24 07:47 style.css
-rw-r--r--. 1 ec2-user ec2-user 3948 Oct 24 07:47 todayDeal.js
[ec2-user@ip-172-31-31-254 Frontend_based_source]$ nano Dockerfile
[ec2-user@ip-172-31-31-254 Frontend_based_source]$ cat Dockerfile
FROM nginx:stable-alpine
COPY . /usr/share/nginx/html/
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
[ec2-user@ip-172-31-31-254 Frontend_based_source]$
i-08ab089125ef328e3 (Terraform-Jenkins-Server)
PublicIPs: 35.173.248.140 PrivateIPs: 172.31.31.254

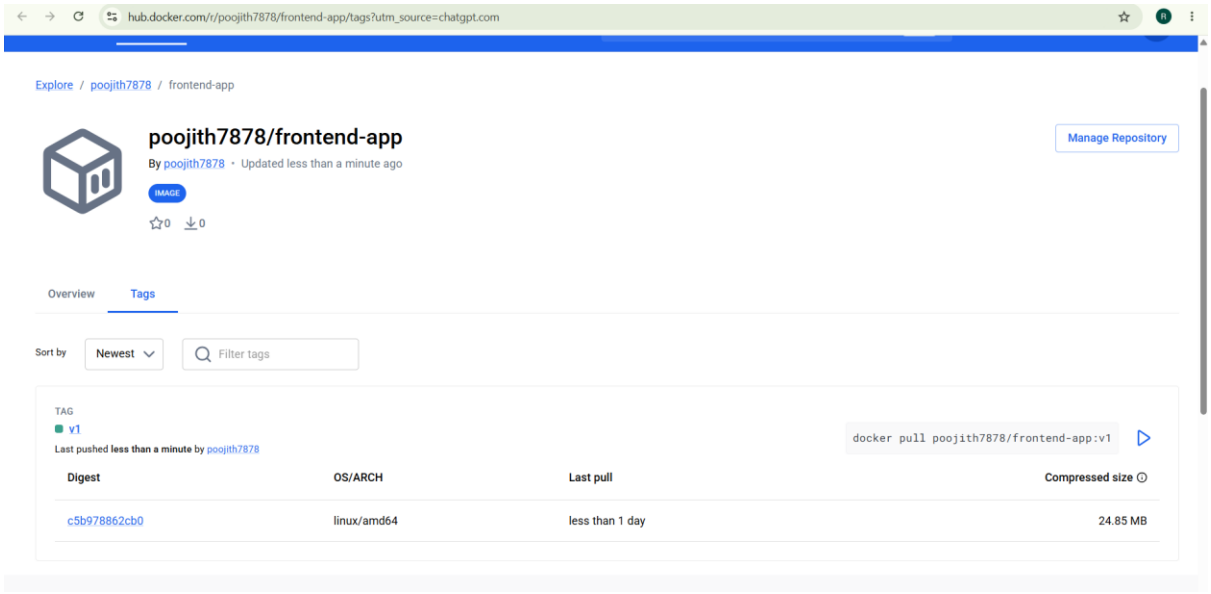

us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=us-east-1&connType=standard&instanceId=i-08ab089125ef328e3&osUser=ec2-user&sshPort=22&addressFamil...
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
CMD ["nginx", "-g", "daemon off;"]
[ec2-user@ip-172-31-31-254 Frontend_based_source]$ sudo docker build -t frontend-app:v1 .
[+] Building 1.8s (7/7) FINISHED
docker:default
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 200B 0.0s
=> [internal] load metadata for docker.io/library/nginx:stable-alpine 0.3s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [internal] load build context 0.0s
=> => transferring context: 39.73kB 0.0s
=> [1/2] FROM docker.io/library/nginx:stable-alpine@sha256:47928bf67eb62a1863cb08d9ecb52d3e704304ba614eaee17884204677702951 1.3s
=> => resolve docker.io/library/nginx:stable-alpine@sha256:47928bf67eb62a1863cb08d9ecb52d3e704304ba614eaee17884204677702951 0.0s
=> => extracting sha256:7b229517ed222597277036d94892173207d01a731ab3a88eeb1de2e963a2c788 0.2s
=> => sha256:47928bf67eb62a1863cb08d9ecb52d3e704304ba614eaee17884204677702951 10.32kB / 10.32kB 0.0s
=> => sha256:88f52f3d58a505b7e1ad96ca6969478517bcccb58418e75a54223271a64da7df 12.29kB / 12.29kB 0.0s
=> => sha256:7b229517ed222597277036d94892173207d01a731ab3a88eeb1de2e963a2c788 3.85MB / 3.85MB 0.1s
=> => sha256:f19a2d3b695178ba0e57d473535f74f5d934f45e191de8334e730f83d2c62704 627B / 627B 0.0s
=> => sha256:885725a63a7cf1d4d5aac105944b35ece2b69a8c2cc6f7040f04727b7b8a63d2 964B / 964B 0.1s
=> => sha256:84ed8c0dd296c188bfc24ed506f46589e9285fb0b11cf3a04d91d524d3067e11 2.49kB / 2.49kB 0.0s
=> => sha256:9f0afb44f10e826253b342c0265e188067c2f0a30c782a52e329928a763adc60 1.88MB / 1.88MB 0.1s
=> => sha256:81b64ae234ef95f3cfc19b7f6c45a511eccbefa6148889ee33ed43c6c3f39939 1.21kB / 1.21kB 0.1s
=> => sha256:bb658a2541968d2b90a8ee165ad08b7f71055c7f7832bdff98e2de2d8b90cf14 1.40kB / 1.40kB 0.1s
=> => sha256:5b8cb54cfa408030aaa43d7bffa9141e0ae6f8a479df13c7aa152100cca358fb 20.31MB / 20.31MB 0.4s


Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
REPOSITORY TAG IMAGE ID CREATED SIZE
frontend-app v1 4d00800d4148 44 seconds ago 62.4MB
[ec2-user@ip-172-31-31-254 Frontend_based_source]$ sudo docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
[ec2-user@ip-172-31-31-254 Frontend_based_source]$ sudo ss -ltnp | grep ':8086'
[ec2-user@ip-172-31-31-254 Frontend_based_source]$ sudo docker run -d --name frontend-app -p 8086:80 frontend-app:v1
b6a27869f3cdf1fc7ab8d60e27d4dfc0bafc928514def9a1e21a1771f5c2ce94
[ec2-user@ip-172-31-31-254 Frontend_based_source]$ sudo docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
b6a27869f3cd frontend-app:v1 "/docker-entrypoint.…" 4 seconds ago Up 4 seconds 0.0.0.0:8086->80/tcp, :::8086->80/tcp frontend-app
[ec2-user@ip-172-31-31-254 Frontend_based_source]$ curl -I http://localhost:8086
HTTP/1.1 200 OK
Server: nginx/1.30.6
Date: Sat, 24 Oct 2026 07:51:22 GMT
Content-Type: text/html
Content-Length: 22657
Last-Modified: Sat, 24 Oct 2026 07:47:39 GMT
Connection: keep-alive
ETag: "6adc629b-5881"
Accept-Ranges: bytes
[ec2-user@ip-172-31-31-254 Frontend_based_source]$
i-08ab089125ef328e3 (Terraform-Jenkins-Server)
PublicIPs: 35.173.248.140 PrivateIPs: 172.31.31.254


us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#SecurityGroup:group-id=sg-034bd23070ed661cd
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
EC2 > Security Groups > sg-034bd23070ed661cd
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
UltraServers New
Capacity Manager New
Images
AMIs
Inbound security group rules successfully modified on security group (sg-034bd23070ed661cd | terraform-jenkins-sg)
Details
956035085469
3 Permission entries
1 Permission entry
Inbound rules
Outbound rules
Sharing
VPC associations
Related resources - new
Tags
Inbound rules (3)
Manage tags
Edit inbound rules
Search
Security group rule ID
IP version
Type
Protocol
Port range
sgr-0cb9c9482bcad5395 IPv4 Custom TCP TCP 8080
sgr-099860049e4fe49c5 IPv4 Custom TCP TCP 8086
sgr-0550e4de3ba534eb7 IPv4 SSH TCP 22


Not secure 35.173.248.140:8086
amazon.in
Hello Select your address
All
English
Hello,sign in Account & Lists
Returns & Orders
Cart
All Best Sellers Today's Deals Mobiles Customer Service Electronic Home & Kitchen Fashion Book Prime
miniTV Bingewatch latest webseries for FREE
Starting ₹99
Cricket world cup festival
4th - 6th November
FREE DELIVERY ON FIRST ORDER
Up to 60% off | Styles for Men
clothing
Footwear
Watches
Bags & language
Redefine your living room
Sofa cum beds
Office chairs & study
Bean bags
Explore all
Top rated, premium quality | Amazon Brands &
Home Products
Furniture
Top rated, premium quality | Amazon Brands &
Home Products
Furniture


us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=us-east-1&connType=standard&instanceId=i-08ab089125ef328e3&osUser=ec2-user&sshPort=22&addressFamil...
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
s://hub.docker.com/ to create one.
You can log in with your password or a Personal Access Token (PAT). Using a limited-scope PAT grants better security and is required for organizations using SSO. Learn more at https://docs.docker.com/go/access-tokens/
Username: poojith7878
Password:
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store
Login Succeeded
[ec2-user@ip-172-31-31-254 Frontend_based_source]$ sudo docker push poojith7878/frontend-app:v1
The push refers to repository [docker.io/poojith7878/frontend-app]
8ce3b221d5b9: Pushed
8ff2a17964b4: Mounted from library/nginx
3212502c73d1: Mounted from library/nginx
2ba21653acee: Mounted from library/nginx
313dac429378: Mounted from library/nginx
d87f6628aa86: Mounted from library/nginx
a8a09d37d441: Mounted from library/nginx
5b91dba2fc11: Mounted from library/nginx
75c137aedfb2: Mounted from library/nginx
v1: digest: sha256:c5b978862cb085b9fcff6ade5df56ffec55141230769e7e1dc78c108194a3117 size: 2197
[ec2-user@ip-172-31-31-254 Frontend_based_source]$



- From the **Java-based source code**, write a Dockerfile, build, run, and push the image to the Docker registry.

Objective: Containerize the Java Spring Boot application, run it in Docker, and upload the image to Docker Hub.

Implementation steps

1. Navigated to the Java application source directory.

cd ~/docker-tasks/Java_based_source

2. Reviewed the application structure and the pom.xml file to identify the Maven build configuration and Java version.
3. Identified that the application used Java 11 and generated an executable JAR.
4. Created the Dockerfile using a multi-stage build.

   FROM maven:3.9-eclipse-temurin-11 AS build

   WORKDIR /app

   COPY pom.xml .

   COPY src ./src

   RUN mvn clean package -DskipTests

   FROM eclipse-temurin:11-jre

   WORKDIR /app

```
COPY --from=build /app/target/spring-boot-web.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

The first stage builds the Java application with Maven. The second stage contains the Java runtime and generated JAR.

5. Built the Java application image.

```
sudo docker build -t java-app:v1 .
```

6. Verified the image.

```
sudo docker images
```

7. Started the container by mapping host port 8087 to application port 8080.

```
sudo docker run -d --name java-app-container -p 8087:8080 java-app:v1
```

8. Verified the container status.

```
sudo docker ps
```

9. Reviewed the application logs.

```
sudo docker logs java-app-container
```

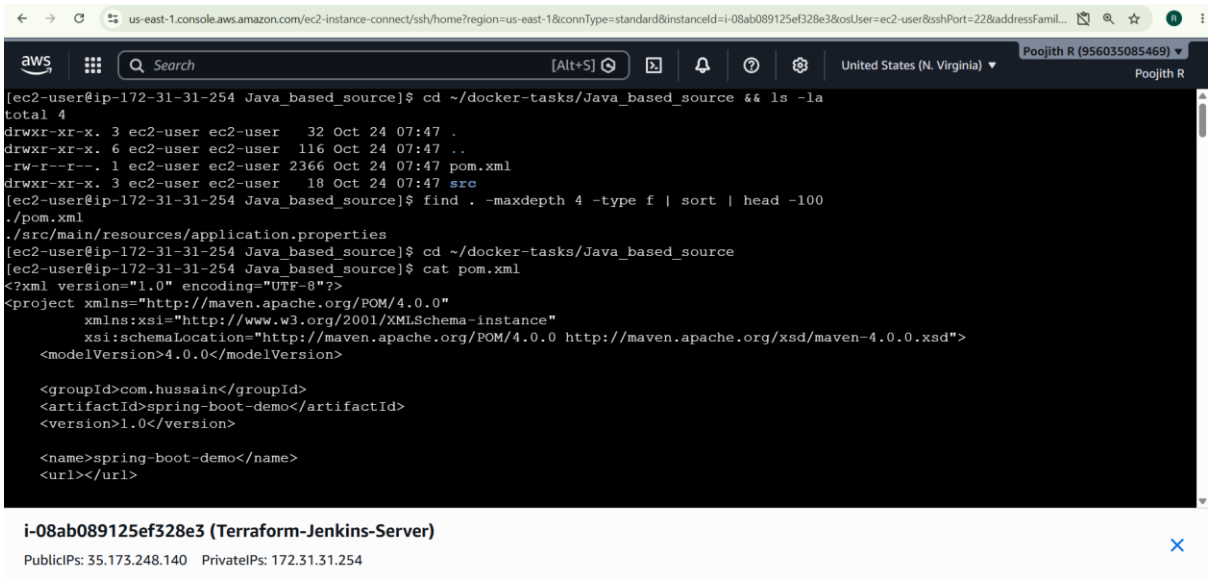
10. Tested the application using the browser URL http://EC2_PUBLIC_IP:8087
11. Tagged the image for Docker Hub.

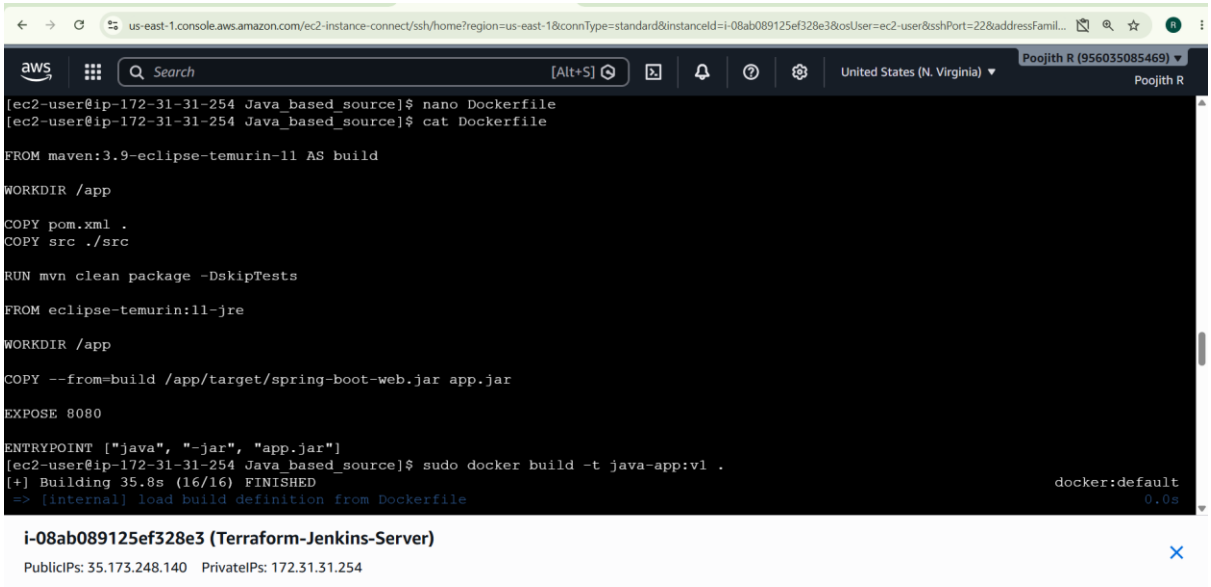
```
sudo docker tag java-app:v1 poojith7878/java-app:v1
```

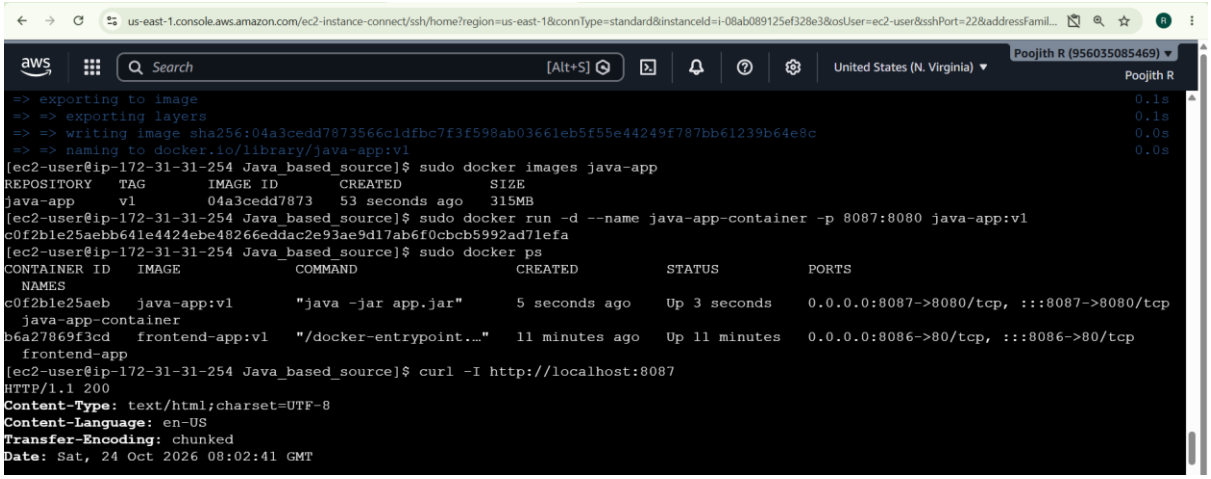
12. Pushed the image to Docker Hub.

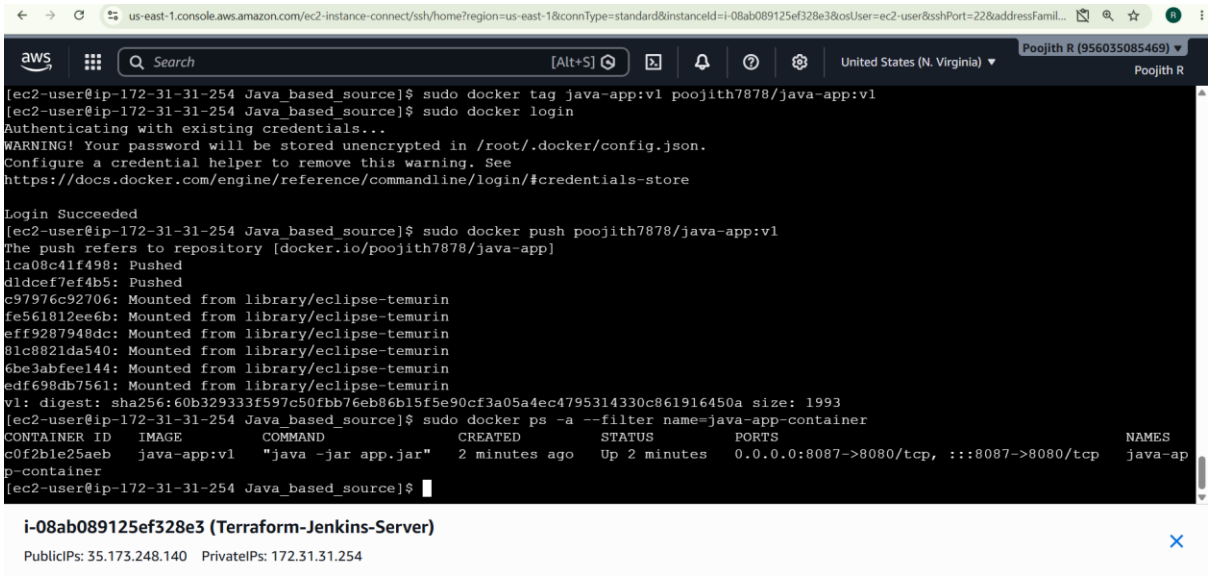
```
sudo docker push poojith7878/java-app:v1
```

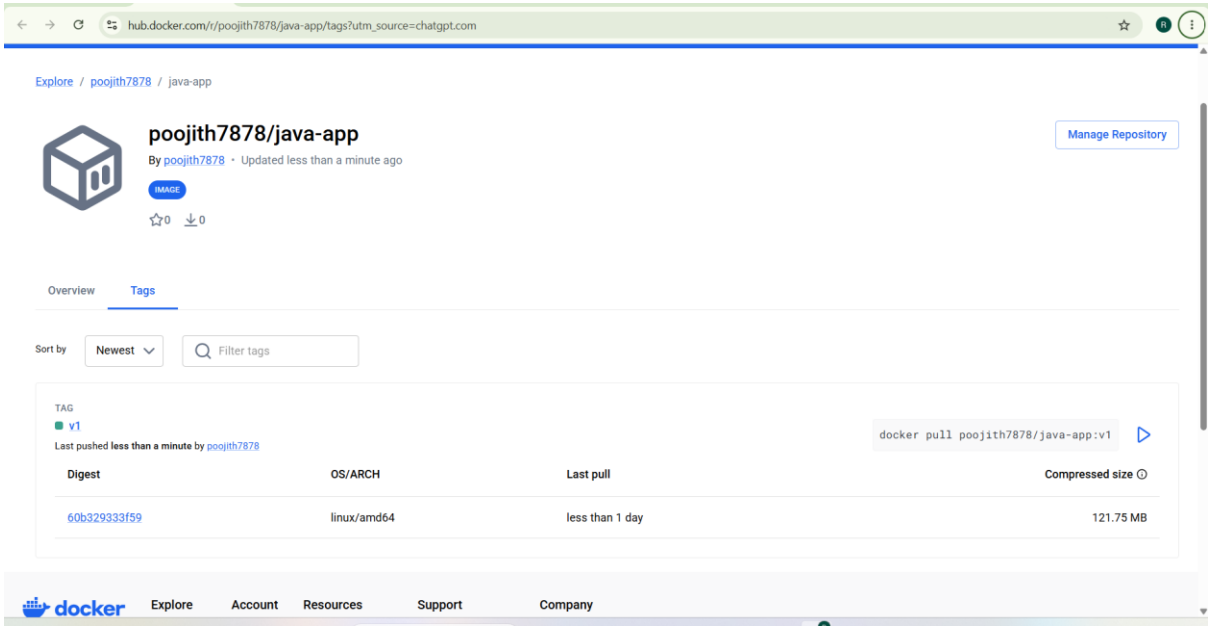
13. Verified that the v1 tag was available.


us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=us-east-1&connType=standard&instanceId=i-08ab089125ef328e3&osUser=ec2-user&sshPort=22&addressFamil...
aws
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
[ec2-user@ip-172-31-31-254 Java_based_source]$ cd ~/docker-tasks/Java_based_source && ls -la
total 4
drwxr-xr-x. 3 ec2-user ec2-user   32 Oct 24 07:47 .
drwxr-xr-x. 6 ec2-user ec2-user 116 Oct 24 07:47 ..
-rw-r--r--. 1 ec2-user ec2-user 2366 Oct 24 07:47 pom.xml
drwxr-xr-x. 3 ec2-user ec2-user   18 Oct 24 07:47 src
[ec2-user@ip-172-31-31-254 Java_based_source]$ find . -maxdepth 4 -type f | sort | head -100
./pom.xml
./src/main/resources/application.properties
[ec2-user@ip-172-31-31-254 Java_based_source]$ cd ~/docker-tasks/Java_based_source
[ec2-user@ip-172-31-31-254 Java_based_source]$ cat pom.xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>
<groupId>com.hussain</groupId>
<artifactId>spring-boot-demo</artifactId>
<version>1.0</version>
<name>spring-boot-demo</name>
<url></url>
i-08ab089125ef328e3 (Terraform-Jenkins-Server)
PublicIPs: 35.173.248.140 PrivateIPs: 172.31.31.254


us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=us-east-1&connType=standard&instanceId=i-08ab089125ef328e3&osUser=ec2-user&sshPort=22&addressFamil...
aws
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
[ec2-user@ip-172-31-31-254 Java_based_source]$ nano Dockerfile
[ec2-user@ip-172-31-31-254 Java_based_source]$ cat Dockerfile
FROM maven:3.9-eclipse-temurin-11 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests
FROM eclipse-temurin:11-jre
WORKDIR /app
COPY --from=build /app/target/spring-boot-web.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
[ec2-user@ip-172-31-31-254 Java_based_source]$ sudo docker build -t java-app:v1 .
[+] Building 35.8s (16/16) FINISHED
docker:default
=> [internal] load build definition from Dockerfile
0.0s
i-08ab089125ef328e3 (Terraform-Jenkins-Server)
PublicIPs: 35.173.248.140 PrivateIPs: 172.31.31.254


us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=us-east-1&connType=standard&instanceId=i-08ab089125ef328e3&osUser=ec2-user&sshPort=22&addressFamil...
aws
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
=> exporting to image
0.1s
=> => exporting layers
0.1s
=> => writing image sha256:04a3cedd7873566c1dfbc7f3f598ab03661eb5f55e44249f787bb61239b64e8c
0.0s
=> => naming to docker.io/library/java-app:v1
0.0s
[ec2-user@ip-172-31-31-254 Java_based_source]$ sudo docker images java-app
REPOSITORY TAG IMAGE ID CREATED SIZE
java-app v1 04a3cedd7873 53 seconds ago 315MB
[ec2-user@ip-172-31-31-254 Java_based_source]$ sudo docker run -d --name java-app-container -p 8087:8080 java-app:v1
c0f2b1e25aebb641e4424ebe48266eddac2e93ae9d17ab6f0cbcb5992ad71efa
[ec2-user@ip-172-31-31-254 Java_based_source]$ sudo docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS
NAMES
c0f2b1e25aeb java-app:v1 "java -jar app.jar" 5 seconds ago Up 3 seconds 0.0.0.0:8087->8080/tcp, :::8087->8080/tcp
java-app-container
b6a27869f3cd frontend-app:v1 "/docker-entrypoint.…" 11 minutes ago Up 11 minutes 0.0.0.0:8086->80/tcp, :::8086->80/tcp
frontend-app
[ec2-user@ip-172-31-31-254 Java_based_source]$ curl -I http://localhost:8087
HTTP/1.1 200
Content-Type: text/html;charset=UTF-8
Content-Language: en-US
Transfer-Encoding: chunked
Date: Sat, 24 Oct 2026 08:02:41 GMT





- From the **Node.js-based source code**, write a Dockerfile, build with tag v1, run, and push it to the Docker registry.

Objective: Containerize the Node.js/React frontend application, run it, and push the image to Docker Hub.

Implementation steps

1. Navigated to the Node.js application source directory.

cd ~/docker-tasks/NodeJs_based_source

2. Reviewed package.json and the package lock file to identify the required dependencies and startup command.
3. Created a file named Dockerfile with the following configuration.

   FROM node:22-alpine

   WORKDIR /app

   COPY package*.json ./

   RUN npm ci

   COPY . .

   EXPOSE 3000

CMD ["npm", "start"]

4. Built the Docker image with the v1 tag.

sudo docker build -t node-app:v1 .

5. Verified that the image was created successfully.

sudo docker images

6. Started the application container using host port 8088 mapped to container port 3000.

sudo docker run -d --name node-app-container -p 8088:3000 node-app:v1

7. Checked the container status and application logs.

sudo docker ps -a --filter name=node-app-container

sudo docker logs node-app-container

8. Tested the application using the browser URL http://EC2_PUBLIC_IP:8088. If required, allowed inbound TCP port 8088 in the EC2 security group. The application must listen on 0.0.0.0 inside the container to be reachable through the port mapping.
9. Logged in to Docker Hub if necessary.

sudo docker login

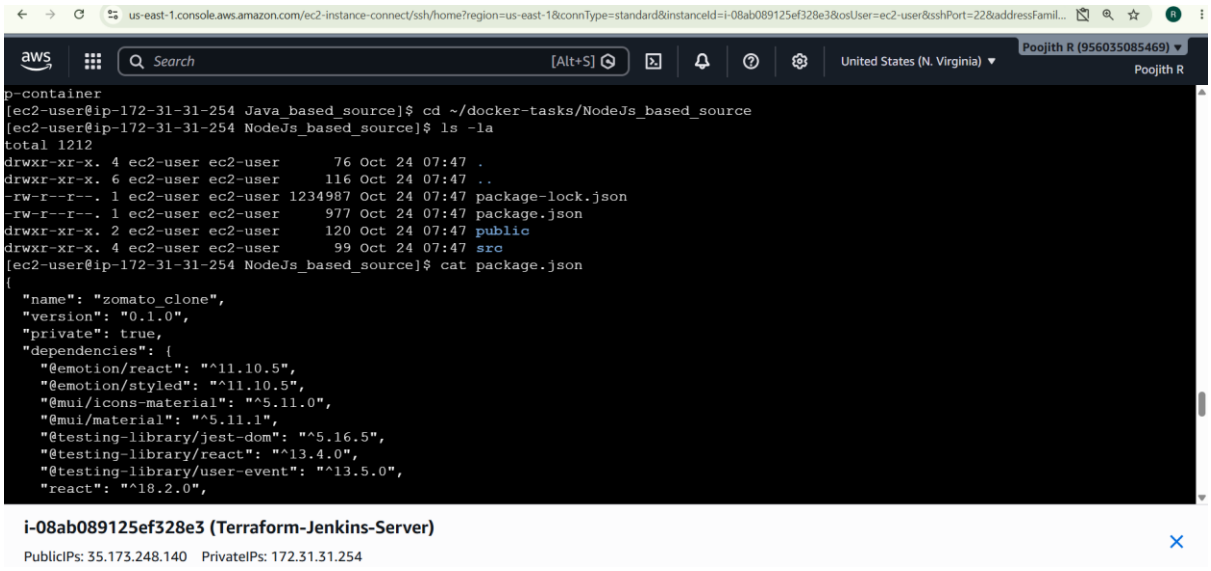
10. Tagged the image with the Docker Hub username and repository name.

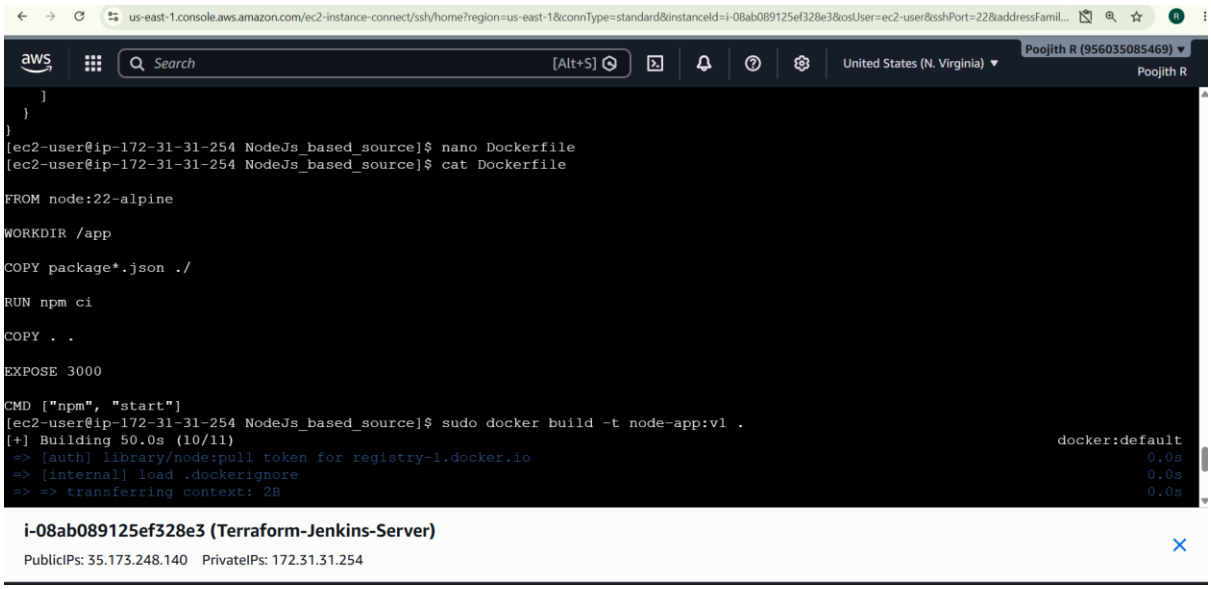
sudo docker tag node-app:v1 poojith7878/node-app:v1

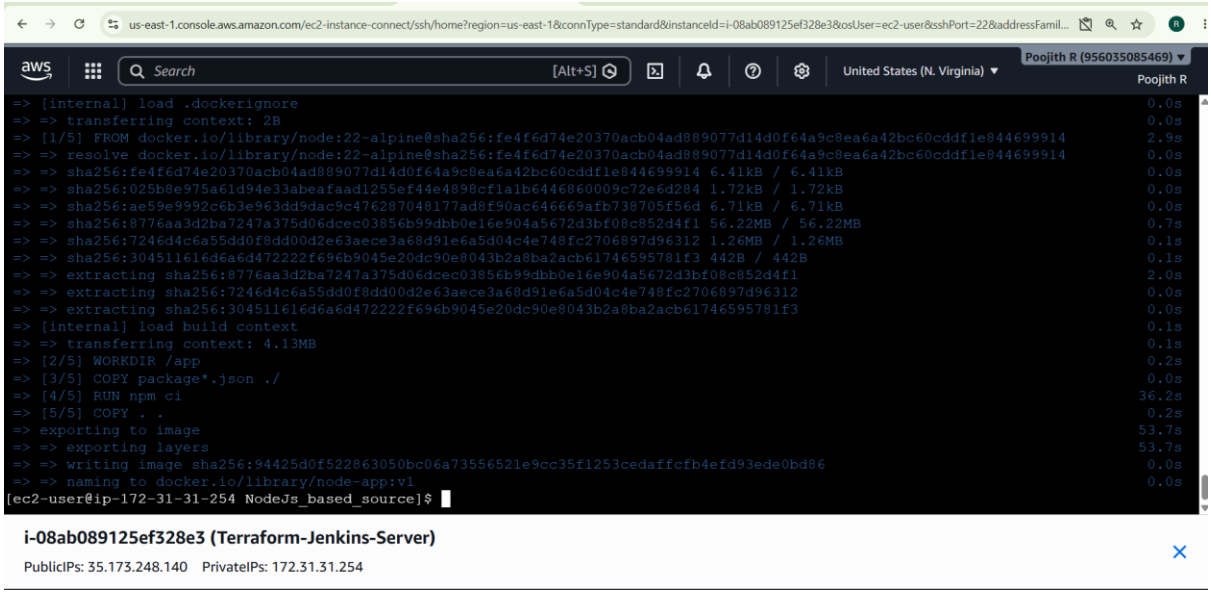
11. Pushed the image to Docker Hub.

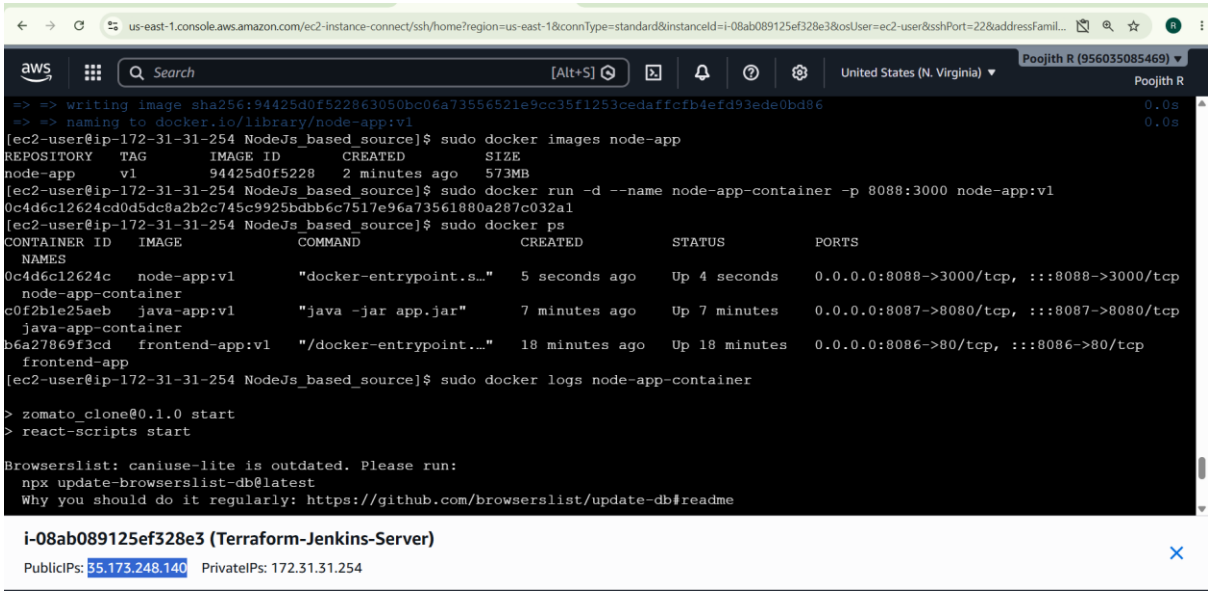
sudo docker push poojith7878/node-app:v1

12. Opened Docker Hub and verified that the v1 tag was available.

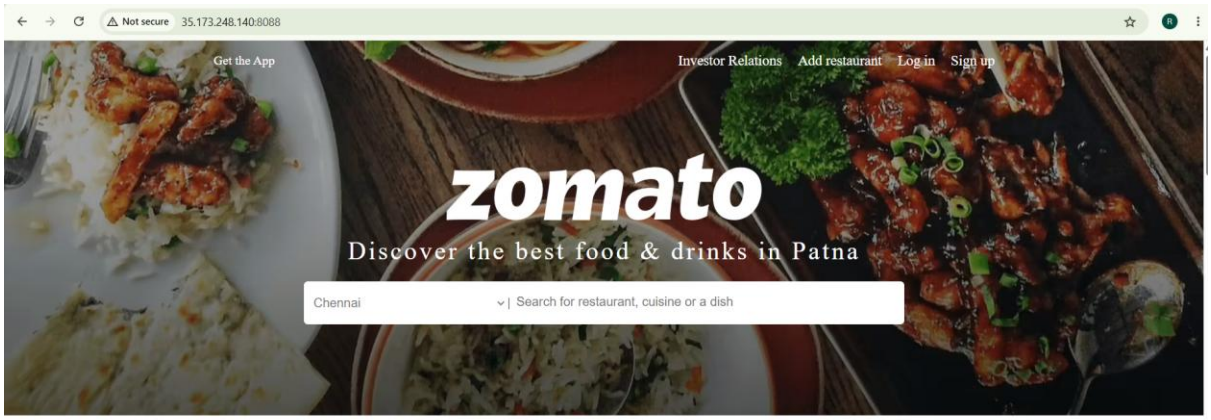





us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=us-east-1&connType=standard&instanceId=i-08ab089125ef328e3&osUser=ec2-user&sshPort=22&addressFamil...
aws
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [1/5] FROM docker.io/library/node:22-alpine@sha256:fe4f6d74e20370acb04ad889077d14d0f64a9c8ea6a42bc60cddf1e844699914 2.9s
=> => resolve docker.io/library/node:22-alpine@sha256:fe4f6d74e20370acb04ad889077d14d0f64a9c8ea6a42bc60cddf1e844699914 0.0s
=> => sha256:fe4f6d74e20370acb04ad889077d14d0f64a9c8ea6a42bc60cddf1e844699914 6.41kB / 6.41kB 0.0s
=> => sha256:025b8e975a61d94e33abeafaad1255ef44e4898cf1a1b6446860009c72e6d284 1.72kB / 1.72kB 0.0s
=> => sha256:ae59e9992c6b3e963dd9dac9c476287048177ad8f90ac646669afb738705f56d 6.71kB / 6.71kB 0.0s
=> => sha256:8776aa3d2ba7247a375d06dcec03856b99dbb0e16e904a5672d3bf08c852d4f1 56.22MB / 56.22MB 0.7s
=> => sha256:7246d4c6a55dd0f8dd00d2e63aece3a68d91e6a5d04c4e748fc2706897d96312 1.26MB / 1.26MB 0.1s
=> => sha256:304511616d6a6d472222f696b9045e20dc90e8043b2a8ba2acb6174659578lf3 442B / 442B 0.1s
=> => extracting sha256:8776aa3d2ba7247a375d06dcec03856b99dbb0e16e904a5672d3bf08c852d4f1 2.0s
=> => extracting sha256:7246d4c6a55dd0f8dd00d2e63aece3a68d91e6a5d04c4e748fc2706897d96312 0.0s
=> => extracting sha256:304511616d6a6d472222f696b9045e20dc90e8043b2a8ba2acb6174659578lf3 0.0s
=> [internal] load build context 0.1s
=> => transferring context: 4.13MB 0.1s
=> [2/5] WORKDIR /app 0.2s
=> [3/5] COPY package*.json ./ 0.0s
=> [4/5] RUN npm ci 36.2s
=> [5/5] COPY . . 0.2s
=> exporting to image 53.7s
=> => exporting layers 53.7s
=> => writing image sha256:94425d0f522863050bc06a73556521e9cc35f1253cedaffcfb4efd93ede0bd86 0.0s
=> => naming to docker.io/library/node-app:v1 0.0s
[ec2-user@ip-172-31-31-254 NodeJs_based_source]$
i-08ab089125ef328e3 (Terraform-Jenkins-Server)
PublicIPs: 35.173.248.140 PrivateIPs: 172.31.31.254

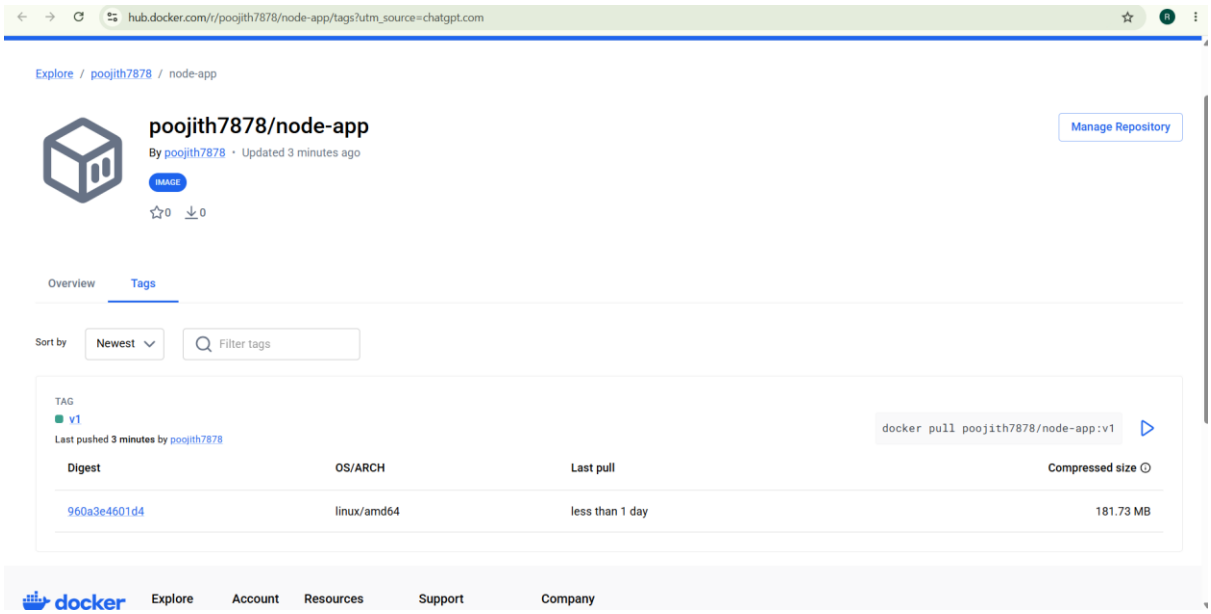

us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=us-east-1&connType=standard&instanceId=i-08ab089125ef328e3&osUser=ec2-user&sshPort=22&addressFamil...
aws
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
=> => writing image sha256:94425d0f522863050bc06a73556521e9cc35f1253cedaffcfb4efd93ede0bd86 0.0s
=> => naming to docker.io/library/node-app:v1 0.0s
[ec2-user@ip-172-31-31-254 NodeJs_based_source]$ sudo docker images node-app
REPOSITORY TAG IMAGE ID CREATED SIZE
node-app v1 94425d0f5228 2 minutes ago 573MB
[ec2-user@ip-172-31-31-254 NodeJs_based_source]$ sudo docker run -d --name node-app-container -p 8088:3000 node-app:v1
0c4d6c12624cd0d5dc8a2b2c745c9925bdbb6c7517e96a73561880a287c032a1
[ec2-user@ip-172-31-31-254 NodeJs_based_source]$ sudo docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS
NAMES
0c4d6c12624c node-app:v1 "docker-entrypoint.s…" 5 seconds ago Up 4 seconds 0.0.0.0:8088->3000/tcp, :::8088->3000/tcp
node-app-container
c0f2b1e25aeb java-app:v1 "java -jar app.jar" 7 minutes ago Up 7 minutes 0.0.0.0:8087->8080/tcp, :::8087->8080/tcp
java-app-container
b6a27869f3cd frontend-app:v1 "/docker-entrypoint.…" 18 minutes ago Up 18 minutes 0.0.0.0:8086->80/tcp, :::8086->80/tcp
frontend-app
[ec2-user@ip-172-31-31-254 NodeJs_based_source]$ sudo docker logs node-app-container
> zomato_clone@0.1.0 start
> react-scripts start
Browserslist: caniuse-lite is outdated. Please run:
npx update-browserslist-db@latest
Why you should do it regularly: https://github.com/browserslist/update-db#readme
i-08ab089125ef328e3 (Terraform-Jenkins-Server)
PublicIPs: 35.173.248.140 PrivateIPs: 172.31.31.254


[ec2-user@ip-172-31-31-254 NodeJs_based_source]$ curl -I http://localhost:8088
HTTP/1.1 200 OK
X-Powered-By: Express
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: *
Access-Control-Allow-Headers: *
Content-Type: text/html; charset=utf-8
Accept-Ranges: bytes
Content-Length: 1711
ETag: W/"6af-+M4OSPFNZpwKBdFEydrj+1+V5xo"
Vary: Accept-Encoding
Date: Sat, 24 Oct 2026 08:10:29 GMT
Connection: keep-alive
Keep-Alive: timeout=5
[ec2-user@ip-172-31-31-254 NodeJs_based_source]$
i-08ab089125ef328e3 (Terraform-Jenkins-Server)
PublicIPs: 35.173.248.140 PrivateIPs: 172.31.31.254





```
[ec2-user@ip-172-31-31-254 NodeJs_based_source]$ sudo docker tag node-app:v1 poojith7878/node-app:v1
[ec2-user@ip-172-31-31-254 NodeJs_based_source]$ sudo docker push poojith7878/node-app:v1
The push refers to repository [docker.io/poojith7878/node-app]
7e5d6ac363f9: Pushed
b29ce89bfaf1: Pushed
3abae0096562: Pushed
556c283b2d2c: Pushed
92ef3b67b69b: Mounted from library/node
7f78e22f72fb: Mounted from library/node
30ad2d1597a9: Mounted from library/node
75c137aedfb2: Mounted from poojith7878/frontend-app
v1: digest: sha256:960a3e4601d461048caceel2c8e773eee28e384d416462f553d5f63f342ac103 size: 1998
[ec2-user@ip-172-31-31-254 NodeJs_based_source]$
```



- Write a **docker-compose file** to set up WordPress with a MySQL database.

Objective: Deploy WordPress and MySQL together using Docker Compose, with persistent database storage.

Implementation steps

1. Connected to the Jenkins EC2 instance and created a project directory.

   mkdir -p ~/wordpress-mysql

cd ~/wordpress-mysql

2. Verified Docker Compose availability. Since Compose was not initially installed, installed the Docker Compose CLI plugin and verified its version.

sudo docker compose version

The command displayed Docker Compose version v5.6.1.

3. Created the docker-compose.yml file.

sudo nano docker-compose.yml

4. Configured the MySQL service with the mysql:5.7 image, database credentials, automatic restart, and a named volume for persistent storage.
5. Configured the WordPress service using the wordpress:latest image, port mapping 8000:80, and database connection settings.
6. Added the following Docker Compose configuration.

   services:

    db:

     image: mysql:5.7

     volumes:

      - db_data:/var/lib/mysql

     restart: always

     environment:

      MYSQL_ROOT_PASSWORD: somewordpress

      MYSQL_DATABASE: wordpress

      MYSQL_USER: wordpress

      MYSQL_PASSWORD: wordpress

```
  wordpress:
   depends_on:
    - db
   image: wordpress:latest
   ports:
    - "8000:80"
   restart: always
   environment:
    WORDPRESS_DB_HOST: db:3306
    WORDPRESS_DB_USER: wordpress
    WORDPRESS_DB_PASSWORD: wordpress
    WORDPRESS_DB_NAME: wordpress


 volumes:
db_data:
```

7. Saved the file and validated the configuration.

```
sudo docker compose config
```

The command displayed the two services, network, and named volume without a configuration error.

8. Started both services in detached mode.

```
sudo docker compose up -d
```

Docker downloaded the images, created the network and volume, and started both containers.

9. Checked the status of the services.

```
sudo docker compose ps
```

The output showed both wordpress-mysql-db-1 and wordpress-mysql-wordpress-1 running. WordPress was mapped to host port 8000.

10. Checked the MySQL logs.

```
sudo docker compose logs --tail=100 db
```

11. Verified that MySQL completed initialization. The logs displayed:

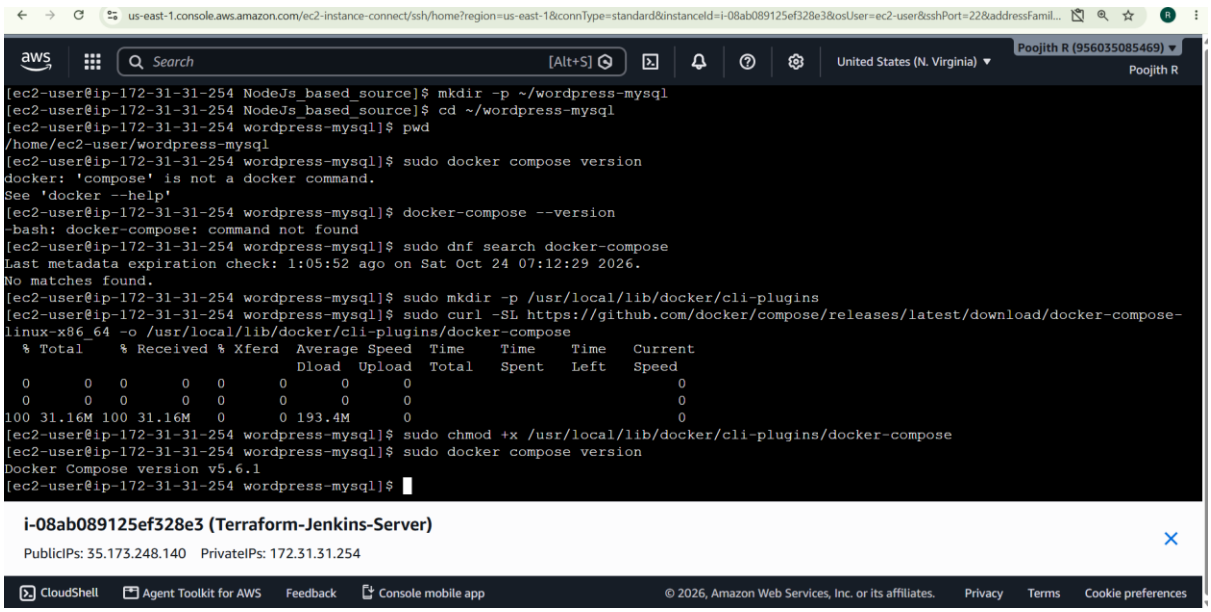
mysqld: ready for connections.

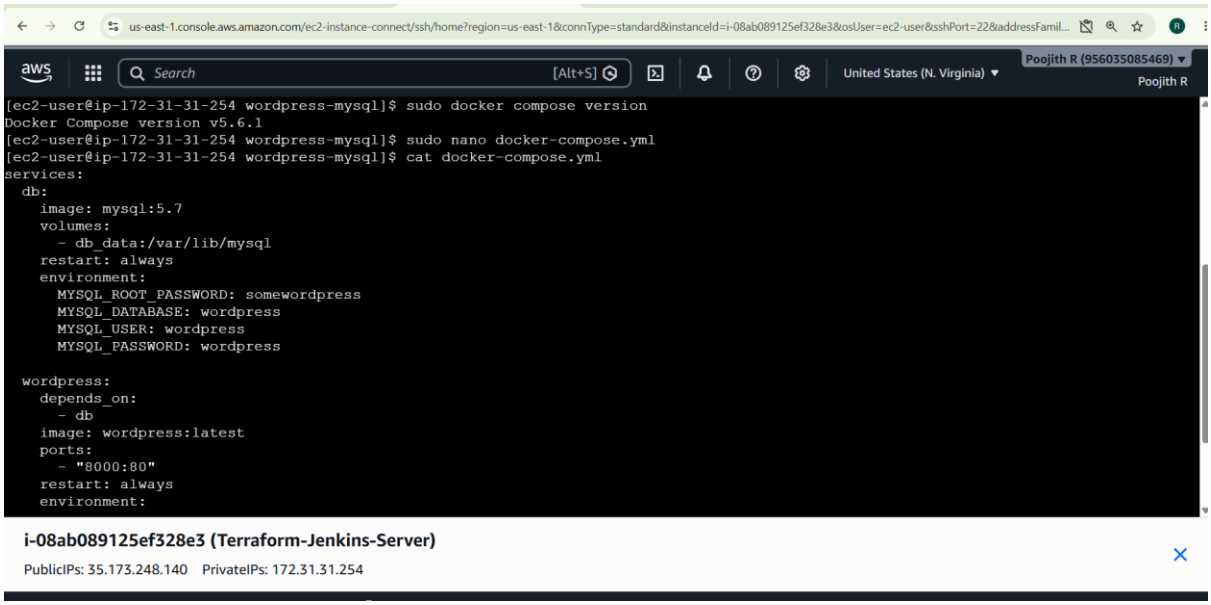
12. Opened the AWS EC2 security group attached to the Jenkins instance and allowed inbound TCP port 8000 from the required client IP.

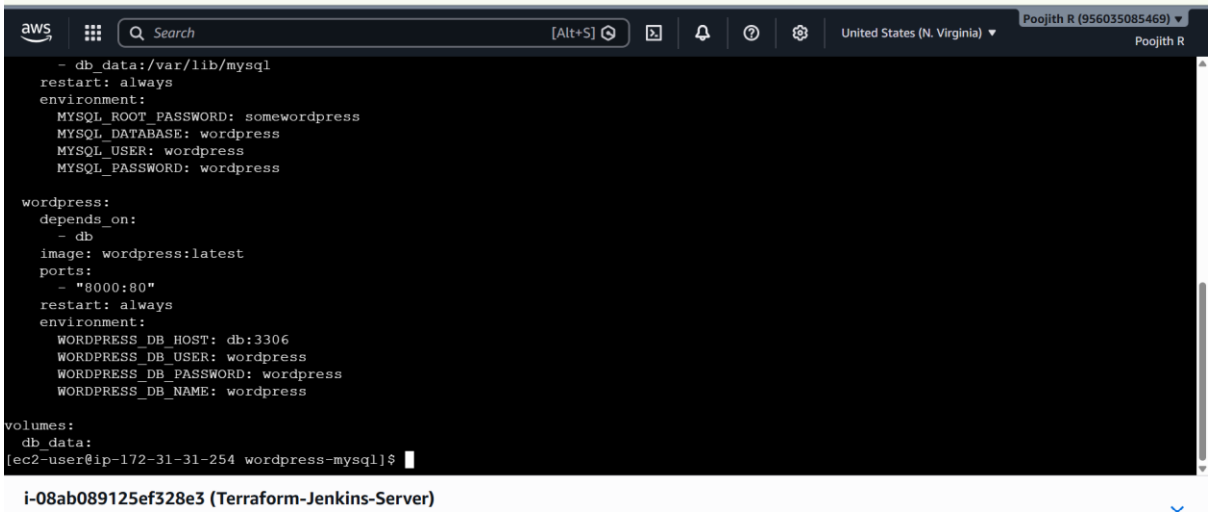
13. Opened the WordPress page in the browser.

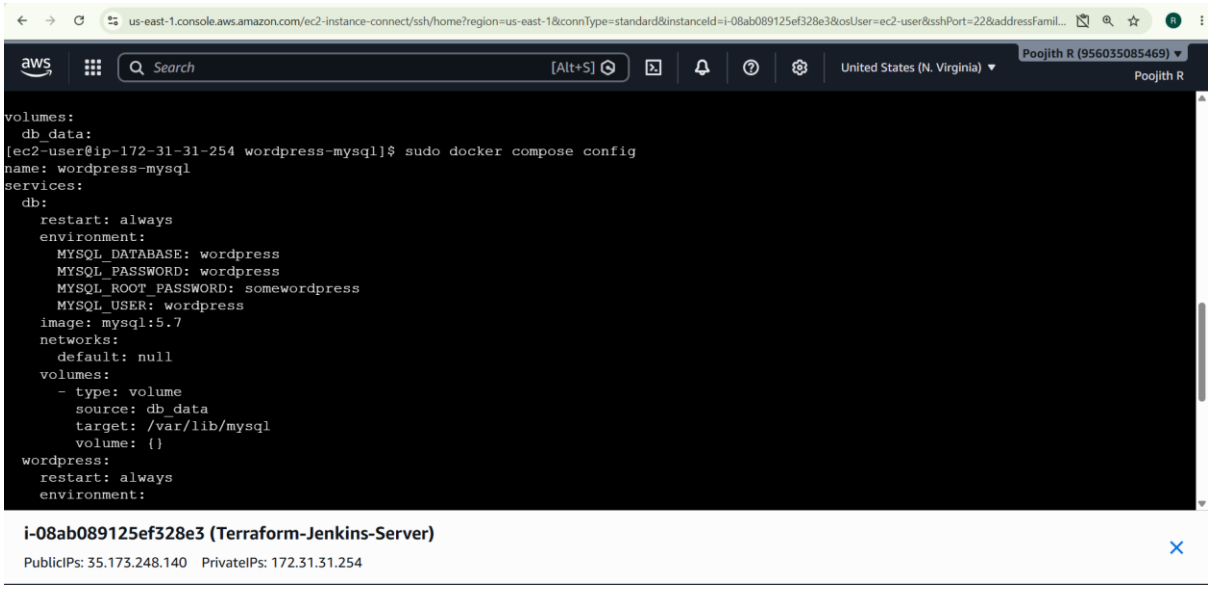
http://EC2_PUBLIC_IP:8000

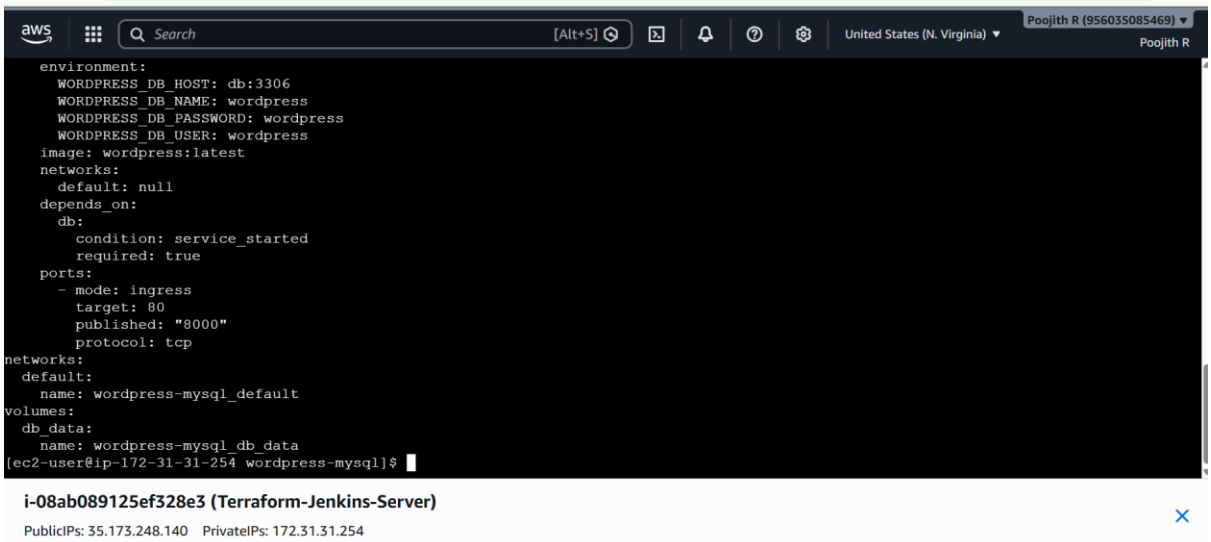
14. Verified that the WordPress installation screen was displayed successfully.






aws
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
- db_data:/var/lib/mysql
restart: always
environment:
MYSQL_ROOT_PASSWORD: somewordpress
MYSQL_DATABASE: wordpress
MYSQL_USER: wordpress
MYSQL_PASSWORD: wordpress
wordpress:
depends_on:
- db
image: wordpress:latest
ports:
- "8000:80"
restart: always
environment:
WORDPRESS_DB_HOST: db:3306
WORDPRESS_DB_USER: wordpress
WORDPRESS_DB_PASSWORD: wordpress
WORDPRESS_DB_NAME: wordpress
volumes:
db_data:
[ec2-user@ip-172-31-31-254 wordpress-mysql]$
i-08ab089125ef328e3 (Terraform-Jenkins-Server)

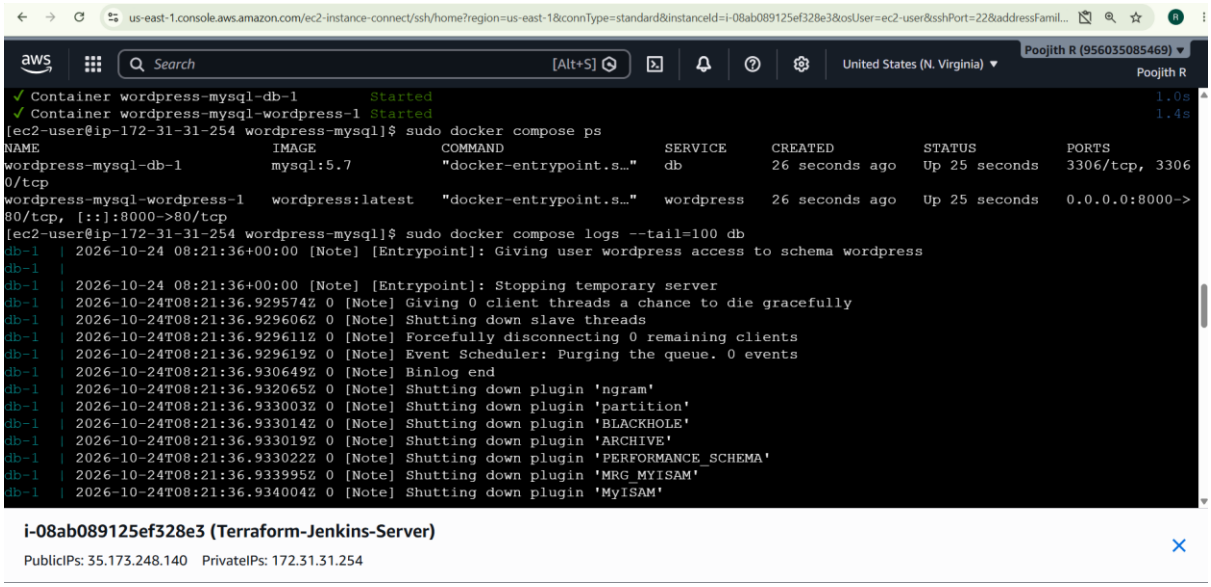

us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=us-east-1&connType=standard&instanceId=i-08ab089125ef328e3&osUser=ec2-user&sshPort=22&addressFamil...
aws
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
volumes:
db_data:
[ec2-user@ip-172-31-31-254 wordpress-mysql]$ sudo docker compose config
name: wordpress-mysql
services:
db:
restart: always
environment:
MYSQL_DATABASE: wordpress
MYSQL_PASSWORD: wordpress
MYSQL_ROOT_PASSWORD: somewordpress
MYSQL_USER: wordpress
image: mysql:5.7
networks:
default: null
volumes:
- type: volume
source: db_data
target: /var/lib/mysql
volume: {}
wordpress:
restart: always
environment:
i-08ab089125ef328e3 (Terraform-Jenkins-Server)
PublicIPs: 35.173.248.140 PrivateIPs: 172.31.31.254


aws
Search
[Alt+S]
United States (N. Virginia)
Poojith R (956035085469)
Poojith R
environment:
WORDPRESS_DB_HOST: db:3306
WORDPRESS_DB_NAME: wordpress
WORDPRESS_DB_PASSWORD: wordpress
WORDPRESS_DB_USER: wordpress
image: wordpress:latest
networks:
default: null
depends_on:
db:
condition: service_started
required: true
ports:
- mode: ingress
target: 80
published: "8000"
protocol: tcp
networks:
default:
name: wordpress-mysql_default
volumes:
db_data:
name: wordpress-mysql_db_data
[ec2-user@ip-172-31-31-254 wordpress-mysql]$
i-08ab089125ef328e3 (Terraform-Jenkins-Server)
PublicIPs: 35.173.248.140 PrivateIPs: 172.31.31.254

```
volumes:
  db_data:
    name: wordpress-mysql_db_data
[ec2-user@ip-172-31-31-254 wordpress-mysql]$ sudo docker compose up -d
[+] up 41/41
 ✓ Image wordpress:latest               Pulled                                    19.6s
 ✓ Image mysql:5.7                      Pulled                                    18.9s
 ✓ Network wordpress-mysql_default      Created                                    0.2s
 ✓ Volume wordpress-mysql_db_data       Created                                    0.0s
 ✓ Container wordpress-mysql-db-1       Started                                    1.0s
 ✓ Container wordpress-mysql-wordpress-1 Started                                   1.4s
[ec2-user@ip-172-31-31-254 wordpress-mysql]$
```

i-08ab089125ef328e3 (Terraform-Jenkins-Server)

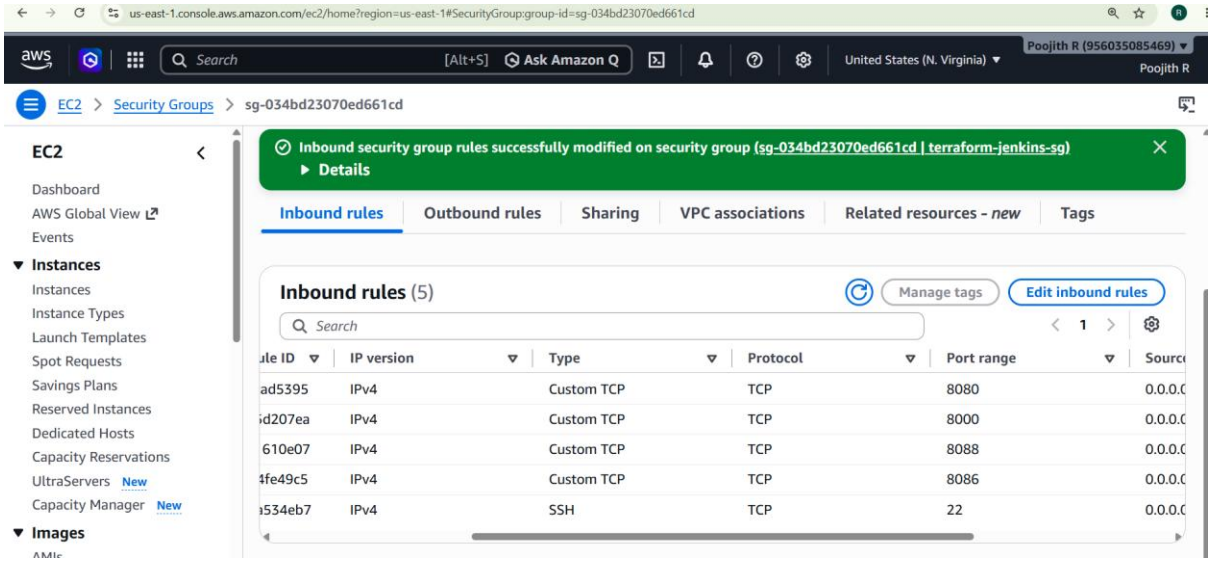
PublicIPs: 35.173.248.140 PrivateIPs: 172.31.31.254

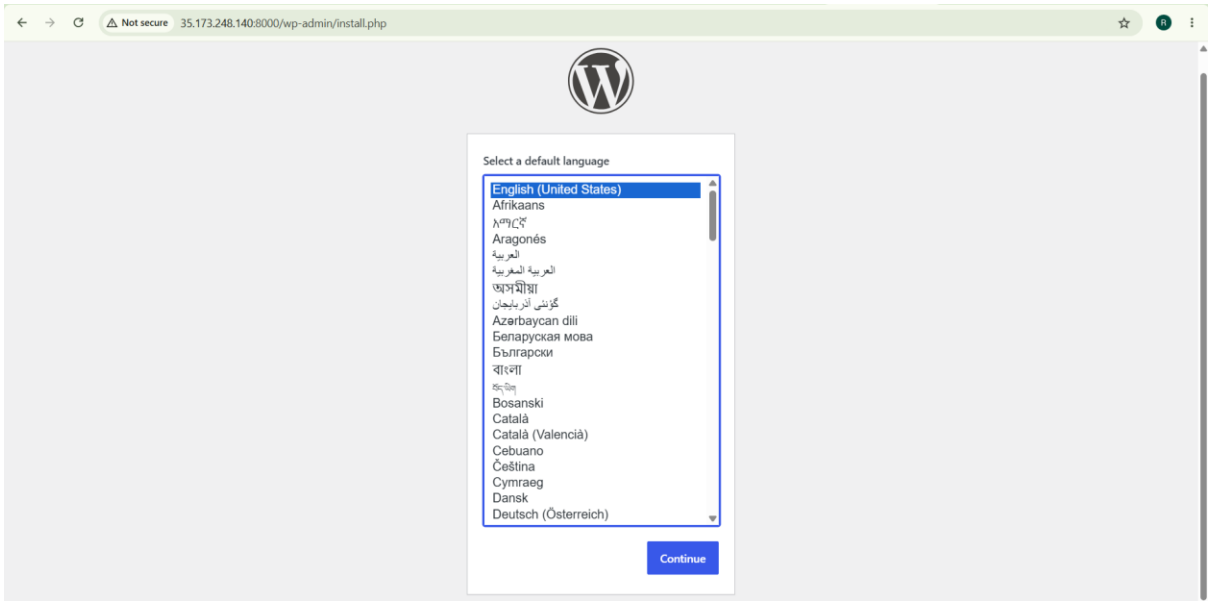


```
 ✓ Container wordpress-mysql-db-1       Started                                    1.0s
 ✓ Container wordpress-mysql-wordpress-1 Started                                   1.4s
[ec2-user@ip-172-31-31-254 wordpress-mysql]$ sudo docker compose ps
NAME                        IMAGE              COMMAND                  SERVICE     CREATED          STATUS          PORTS
wordpress-mysql-db-1        mysql:5.7          "docker-entrypoint.s…"   db          26 seconds ago   Up 25 seconds   3306/tcp, 3306
0/tcp
wordpress-mysql-wordpress-1   wordpress:latest   "docker-entrypoint.s…"   wordpress   26 seconds ago   Up 25 seconds   0.0.0.0:8000->
80/tcp, [::]:8000->80/tcp
[ec2-user@ip-172-31-31-254 wordpress-mysql]$ sudo docker compose logs --tail=100 db
db-1  | 2026-10-24 08:21:36+00:00 [Note] [Entrypoint]: Giving user wordpress access to schema wordpress
db-1  |
db-1  | 2026-10-24 08:21:36+00:00 [Note] [Entrypoint]: Stopping temporary server
db-1  | 2026-10-24T08:21:36.929574Z 0 [Note] Giving 0 client threads a chance to die gracefully
db-1  | 2026-10-24T08:21:36.929606Z 0 [Note] Shutting down slave threads
db-1  | 2026-10-24T08:21:36.929611Z 0 [Note] Forcefully disconnecting 0 remaining clients
db-1  | 2026-10-24T08:21:36.929619Z 0 [Note] Event Scheduler: Purging the queue. 0 events
db-1  | 2026-10-24T08:21:36.930649Z 0 [Note] Binlog end
db-1  | 2026-10-24T08:21:36.932065Z 0 [Note] Shutting down plugin 'ngram'
db-1  | 2026-10-24T08:21:36.933003Z 0 [Note] Shutting down plugin 'partition'
db-1  | 2026-10-24T08:21:36.933014Z 0 [Note] Shutting down plugin 'BLACKHOLE'
db-1  | 2026-10-24T08:21:36.933019Z 0 [Note] Shutting down plugin 'ARCHIVE'
db-1  | 2026-10-24T08:21:36.933022Z 0 [Note] Shutting down plugin 'PERFORMANCE_SCHEMA'
db-1  | 2026-10-24T08:21:36.933995Z 0 [Note] Shutting down plugin 'MRG_MYISAM'
db-1  | 2026-10-24T08:21:36.934004Z 0 [Note] Shutting down plugin 'MyISAM'
```

i-08ab089125ef328e3 (Terraform-Jenkins-Server)

PublicIPs: 35.173.248.140 PrivateIPs: 172.31.31.254




Not secure
35.173.248.140:8000/wp-admin/install.php
Select a default language
English (United States)
Afrikaans
አማርኛ
Aragonés
العربية
العربية المغربية
অসমীয়া
گؤنئی آذربایجان
Azərbaycan dili
Беларуская мова
Български
বাংলা
ཇོང་ཁ
Bosanski
Català
Català (Valencià)
Cebuano
Čeština
Cymraeg
Dansk
Deutsch (Österreich)
Continue